Changelog — vasic-digital/tmux

All releases use Semantic Versioning. Every release carries a positive-runtime-evidence verification record per the project's anti-bluff covenant (Constitution §101 / universal §11.4).

[tmux-1.0.40] (also v1.0.40) — 2026-08-11

Fixed

- **Split-topology (`TMX_SERVER_SPLIT=1`) crash-isolation tests failed to engage the split at all — missing `TMX_CPU=auto` opt-in (TMX-082).** Root cause (systematic-debugging, triggered by an operator-reported `install.sh` verification-gate RED): new crash-isolation test coverage for the opt-in server/workload scope-split topology (test 09's T7, test 15's T7-T9) landed on `main` via a separate work stream, one day after v1.0.39. Those new tests spawn `TMX_SERVER_SPLIT=1` sessions without also setting `TMX_CPU` — but since v1.0.39 (TMX-079) `TMX_CPU` defaults to empty/unlimited, and the split topology's `_split_cpu_pcts` requires an explicit, splittable total to engage at all. Without an opt-in `TMX_CPU`, the wrapper correctly falls back LOUDLY to the shared topology ("`TMX_CPU='' is not splittable`") exactly as designed — so no workload slice is ever created, and the new tests' assertions that the split engaged fail. Same root cause + same fix pattern already applied to test 87's G6/G8 sub-checks during the v1.0.39 review cycle, just not yet propagated to this newer, independently-landed test coverage. **Not a regression in the v1.0.39 fix** — confirmed via direct reproduction: the wrapper's own fallback message fires exactly as intended, proving the opt-in mechanism itself is working correctly; the newly-added tests simply hadn't been reconciled to it yet (§11.4.120). Fixed by adding `TMX_CPU=auto` to the four affected spawn commands (`scripts/tests/09_crash_isolation_scope.sh` T7a/T7b, `scripts/tests/15_per_session_cgroup_distinct.sh` T7). Verified: test 09 PASS=20/FAIL=0/SKIP=0 (was 17/1/0), test 15 PASS=11/FAIL=0/SKIP=0 (was previously failing T7/T8).

- **Host-local `~/.config/opencode/opencode.json` (this host only, not tracked in the repo) was missing the codegraph MCP server entry** that every other configured CLI agent (Claude Code, Kimi, Crush, Qwen Code) already had wired on this host — test 22's T2 flagged this pre-existing gap. Not a code defect; a host-configuration gap closed for parity by adding the `codegraph` entry to the `mcp` object using OpenCode's own local-MCP schema (`{"type": "local", "command": ["codegraph", "serve", "--mcp"], "enabled": true}`), verified via a JSON round-trip proving no other one of the file's 132 pre-existing entries was disturbed.

Verification

Reproduced via direct test invocation (`bash scripts/tests/09_...sh`, `15_...sh`, `22_...sh`) before any fix, confirming the exact failure signatures reported by the operator's `install.sh` run; root-caused with a direct minimal reproduction of the wrapper's own fallback path; fixed; re-verified all three tests individually GREEN; full `scripts/verify.sh` gate re-run clean before release.

[tmux-1.0.39] (also v1.0.39) — 2026-08-10

Fixed

- **No resource or lifetime limit by default — every cap is strictly opt-in (TMX-079).** Root-cause forensic anchor (operator report, 2026-08-10): "sessions and processes get killed and wiped out ... on powerful hardware with enough resources". Systematic-debugging traced this to THREE unconditional-by-default caps: (1) the idle-timeout session recycler (`scripts/tmx-recycler.sh`), wired into EVERY session by default, tearing down (kill-session + scope-stop) ANY session with no client attached for >= 900 s — using `#{session_attached}==0` as its SOLE idle signal, deliberately NOT `#{session_activity}`, so a genuinely-active DETACHED background/autonomous job (the normal way to run long tmx work) was killed regardless of whether it was doing anything — the DOMINANT cause and the only mechanism in this codebase that actually kills an already-running session's processes irrespective of host capacity; (2) `CPUQuota` applied to every scope unconditionally (host-adaptive since v1.0.37, but with no off switch); (3) a hardcoded `TasksMax=4096` per scope with NO override knob at all, unlike `TMX_MEM`/`TMX_CPU` (a large multi-agent fleet on a powerful host can legitimately need many thousands of threads/processes — see universal Constitution §12.12). All three now default to fully elastic — the same "liquid" model memory already correctly used — and

are opt-in only: `TMX_CPU` / `TMX_TASKS` unset default to unlimited (`auto` opts IN to the previous host-adaptive/legacy-fixed value, an explicit numeric value opts IN to an explicit cap); `TMX_RECYCLE_IDLE_SECS` unset/`0` defaults to never-auto-recycled (`TMX_RECYCLE_IDLE_SECS=<secs>` opts IN). Darwin's `TMX_CPU_HARD_SEC` / `TMX_PROC_MAX` default to `unlimited` for the same reason (were fixed 24 h / 4096 defaults with no way to remove them). Implemented in `scripts/tmx.template` + `scripts/tmx-recycler.sh`. Acceptance: test 88 (new) proves a session spawned with none of the three knobs set reads back `cpu.max=max` AND `pids.max=max` on its live cgroup AND installs no idle-recycle tmux hook, while a session with all three explicitly opted in still lands the bounded values (regression coverage for the preserved opt-in code paths). Tests 15, 86, and 87 reconciled to the new default (§11.4.120 — a correct fix legitimately changing a prior-asserted default is NOT a fix to revert nor a gate to fake-pass; the gates were rewritten to assert the new mechanism).

Tests

- **Test 88 (new)** — `88_no_limits_by_default.sh`: §11.4.115 RED/GREEN polarity; RED reproduces the three unconditional-cap defaults on the pre-fix artifact, GREEN proves (functionally + live cgroup/hook readback) that every cap is opt-in only and that the opt-in paths still work when explicitly requested.
- **Test 86 reconciled** — CPU truth-table + live-spawn assertions updated from "host-adaptive quota is the default" to "unlimited by default, `TMX_CPU=auto` opts in to the host-adaptive value".
- **Test 87 reconciled** — the two sub-checks (G6 burst, G8 dashed-name escaping) that relied on `TMX_CPU` defaulting to `auto` to engage the opt-in server/workload scope-split topology now pass `TMX_CPU=auto` explicitly.
- **Test 15 reconciled** — T4 now asserts `cpu.max=max` (no `CPUQuota`) on a default session's live cgroup, mirroring T3's existing `memory.max=max` assertion.
- **Meta-test (`meta_test_false_positive_proof.sh`)** — `M-CPUADAPT` fixed (its sed pattern could no longer match the changed default — a silently-escaping paired mutation, itself a §1.1 gap, closed in the same commit) and repointed at the new default; new paired mutation `M-NOLIMITS` reverts the idle-recycle-off-by-default and TasksMax-opt-in changes and proves test 88 catches the regression.

Known pre-existing issues surfaced during this cycle (tracked, NOT caused by this fix — confirmed via A/B isolation against the v1.0.38 baseline, §11.4.114):

- **TMX-080** — test 27 (`27_state_persistence.sh`) sub-check "18" (the `run-shell` cwd-record hook) fails deterministically on iteration 1, in isolation, independent of full-suite load —

reproduced 3× in isolation on an otherwise-idle host, proving the file's own comment attributing it to "full-suite CPU contention" is an incomplete diagnosis. Reproduces identically on the v1.0.38 baseline.
- **TMX-081** — test 87 G4 ("quiet-phase control") occasionally fails to observe a zero-throttle settle window before the load phase begins. Reproduces identically on the v1.0.38 baseline; the isolation invariant it precedes (G5) still passes, proving the actual feature under test is sound.

[tmux-1.0.38] (also v1.0.38) — 2026-07-24

> **§11.4.151 compliance:** From this release onward, annotated release tags use the `tmux-` project-name prefix. The bare `v1.0.38`, `v1.0.37`, and future `v*` tags remain as lightweight legacy aliases; the annotated `tmux-*` tags are the canonical release artifacts.

[v1.0.38] — 2026-07-24

Fixed

- **Scope-independent collision guard (TMX-077).** The session-scope collision guard in `tmx launch` fell back silently when systemd was absent, allowing two sessions with the same name to coexist (and their scopes to collide). The guard now falls back to a PID-backed mutual-exclusion lock when `systemd-run --user` is unavailable, so session-name uniqueness is enforced across the host regardless of the init system. The fallback path is proven via a dedicated test that injects a fake `systemd-run` returning `UnknownObject` and verifies the PID-path gate fires correctly (`tests/86_collision_guard_no_systemd.sh`).

Tests

- **§1.1 meta-test mutations for local-dependency tests (TMX-076).** Pairs mutations for tests 72 (`obtain_local_deps`) and 73 (`build_native` local-deps) — each mutation strips an invariant and the paired test FAILs, proving the gates are not bluff gates. Mutations: `M72` (`install.sh` unreachable), `M73a` (dep-writer write-only), `M73b` (remove `nix-shell` from path).

[v1.0.37] — 2026-07-24

Added

- **Host-adaptive CPUQuota default.** The previous FIXED `CPUQuota=200%` (2 cores) default squeezed every session's entire process tree — tmux server plus all subagents — into 2 cores regardless of host size, causing progressive typing lag and timer stalls in long sessions. Now `TMX_CPU=auto` (the new default) computes a host-adaptive quota: 60% of host cores divided across 4 concurrent sessions (= 15% per core), floored at 200%. On a 64-core host this yields 960% instead of 200%, dropping server round-trip max latency 50 ms→3 ms and eliminating CFS throttling entirely. Small hosts keep the legacy 200% floor. Implemented in `scripts/tmx.template` (`_default_cpu_pct`).
- **Interactive server-scope split (TMX_SERVER_SPLIT=1).** Opt-in topology (default OFF): the tmux SERVER keeps its own lightweight `tmx-NAME.scope` with a small host-adaptive quota, while every pane shell + its child processes run in per-pane transient scopes under a dedicated `tmxw-<name>.slice` carrying the remainder of the session budget plus the burst bank. A bursting fleet can no longer freeze the server's input/echo/timer handling — the split guarantees the operator's keystrokes stay responsive regardless of workload. Pre-conditions: Linux + systemd ≥ 230 + `tmx-pane-shim.sh` installed. Fail-closed: any precondition miss falls back LOUDLY to the proven shared-scope topology. Implemented in `scripts/tmx.template` (`_srv_cpu_pct`, `_split_cpu_pcts`, `_slice_unit_for`) and the new `scripts/tmx-pane-shim.sh` + `scripts/tests/87_server_scope_split.sh`.
- **TMX_CLASSIFICATION export.** Every session now exports `TMX_CLASSIFICATION=tmx-supported` (systemd ≥ 230 with functional `--user --scope`, or Darwin with rlimit) or `TMX_CLASSIFICATION= tmx-degraded` (any other) so downstream tooling can probe session capability programmatically.
- **OOM score adjustment.** Linux tmux server processes now get `oom_score_adj=-500` (via the external `oom-score-helper` when present, or a direct proc write when running as root) so the server that maintains the operator's session is killed last under memory pressure.

Fixed

- **Stale Issues.md statuses rectified (TMX-072 through TMX-075).** All four items — wizard random-suffix (TMX-072), masked password input (TMX-073), single-prompt reopen fix (TMX-074), and existing-session picker (TMX-075) — were marked `Status: Queued` despite being IMPLEMENTED in v1.0.34 (commit `cb3e96c`). Status lines updated to the correct

§11.4.33 closure vocabulary (`Implemented` / `Fixed (→ Fixed.md)`) and entries migrated to Fixed.md with full 4-layer captured-evidence citations.

Verification

- New test `87_server_scope_split.sh` (309 lines, §11.4.115 RED/GREEN polarity) validates the scope split topology end-to-end: throttle telemetry, per-pane scope creation/teardown, fail-closed fallback, and deterministic 3× consistency.
- Host-adaptive CPUQuota defaults validated via the running fleet (forensic: 200%→960% dropped throttle from 18.8% to 0% on a 64-core host; server round-trip latency 50 ms→3 ms).
- `setup.sh` + `run_all.sh` full suite retest pending at tag creation.

[v1.0.35] — 2026-07-05

Added

- **Session-name sanitization for spaces and special characters.** Both the `tmx new -s NAME` operator path and the interactive wizard prompt now normalize a typed/passed name instead of rejecting it: leading/trailing whitespace is stripped, internal whitespace runs are collapsed to a single `-`, and any remaining characters outside the safe session-name set are deleted. Names like `"hello world"` become `hello-world`, so the operator can type naturally without memorizing the allowed character set. Empty or whitespace-only input continues to fall through to the existing empty/default picker path (no accidental session creation). Implemented in `scripts/tmx.template` and `scripts/tmx-shell-init.sh.template`.
- **Wizard colour syntax preserved alongside sanitization.** The interactive prompt keeps `:` and `#` so `name:red` and `name:#hex` still reach the wrapper; the random suffix is inserted before the colour token (`home-1234:red`, not `home:red-1234`) so the wrapper sees a valid colour.

Fixed

- **Test reconciliation for the intentional safe-set change.** Historical tests 63 (escaped-colour syntax) and 65 (wizard prompt validation) asserted the old reject-on-invalid behaviour; they now assert the new normalize- then-forward behaviour per §11.4.120, preventing a false RED after the sanitization change.

Verification

- New test `35_session_name_validation.sh` covers the wizard sanitization path (spaces, tabs, special characters, trim, empty/whitespace-only, colour-preservation) with a fake `tmx` and 3× determinism.
- New test `82_session_name_sanitization_live.sh` creates real tmux sessions with messy names and verifies the sanitized name and socket label match.
- Paired meta-test `M82` neutralizes the whitespace-to-hyphen handling in the generated `scripts/tmx`; test 82 fails, proving the guard has teeth.
- `setup.sh`: **PASS=69 FAIL=0 SKIP=13** (nezha, Linux x86_64).
- Standalone `run_all.sh`: **PASS=69 FAIL=0 SKIP=13** (nezha, Linux x86_64).
- `meta_test_false_positive_proof.sh`: **66 mutations caught, 0 escaped, 11 skipped** (layer-4 coverage active). `M82` was skipped by the automated harness (mutation-target lookup transient); it was manually reproduced: applying the mutation makes test 82 FAIL, restoring makes test 82 PASS — the guard has teeth.
- Live operator-path retest: `tmx new -s 'hello world' -d` created `hello-world`; `tmx new -s '  messy! name  ' -d` created `messy-name`; `tmx new -s 'hello world:red' -d` created `hello-world` with `status-style bg=red`. All sessions deleted cleanly.

[v1.0.34] — 2026-07-05

Added

- **Random 4-digit suffix on wizard-created sessions.** Typing a session name at the interactive `tmx` wizard now always creates a session whose real name is the typed name plus a random 4-digit suffix (e.g. `my-session-2507`), so retyping the same base name can never collide with an earlier session. Set `TMX_EXACT_NAME=1` to opt out for scripted/deterministic use. Suffix generation mixes the process PID into the `awk` seed alongside wall-clock time to remove a same-second collision window across concurrent processes (test 78, meta-test `M-SUFFIX`).
- **Existing-session picker on blank wizard input.** Pressing Enter with no typed name at the wizard no longer always drops to a bare shell. If any sessions already exist, the operator sees a numbered list plus a `0) None` option and can join one directly (still prompted for its password exactly once if protected); choosing `None` (or pressing Enter again) keeps the original bare-shell behaviour (test 79, meta-test coverage via the same suite).
- **Masked password input.** Every password prompt in the `tmx` wrapper now echoes `*` per keystroke (with working backspace)

instead of plaintext, via a shared `_read_password_masked` helper (test 77, meta-test `M-MASK`).

Fixed

- **Reopening a password-protected session no longer asks for the password twice.** Root cause: the `attach` verb checked the remembered password BEFORE checking whether the session was actually still running, so a doomed attach on an idle-recycled session fell through to the create flow, which unconditionally asked to set a password again — looking like it might be resetting the password (it wasn't; typing the same password both times always worked, but the UX was confusing and unsafe-looking). Opening an already-protected session — live or idle-recycled — now verifies the password exactly ONCE; only a genuinely brand-new session name asks for a password AND a confirmation, with a fail-closed 3-attempt-mismatch abort that leaves no session or password behind (tests 81, 84, 80; meta-test `M-LIVEFIRST`/`M-CONFIRM`).
- **`workable-items validate` `TMX-071` sequence-gap** (pre-existing, unrelated to the above): a June 30 revert of an orphan DB row (tooling gap, not a content defect) left a permanent gap in the append-only TMX-NNN sequence. Retired honestly as an `Obsolete` tombstone citing both commits; the still- undone follow-up work it tracked is re-filed fresh as `TMX-076` so nothing is silently lost.

Verification

Independent whole-branch code review + a full regression sweep (`setup.sh --verify-only`, PASS=68 FAIL=0 SKIP=13) caught and closed 5 additional findings before this release: a critical meta-test bluff-gate (4 mutations targeting the never-rebuilt `.template` source with a tautological `FAIL` regex, rewritten to target the generated artefact with precise per-test patterns), the suffix-collision hardening above, a variable-namespace leak into the operator's sourced shell, a broken test-only liveness marker, and a second real regression found during the sweep itself (test 54's sandbox never isolated `TMUX_TMPDIR`, so the new picker diverted into a session- choice prompt on any host with live sessions, masking the double-prompt idempotency guard). All fixed and re-verified 3x deterministic. Installed + retested live on host `nezha` (Linux x86_64) alongside 6 pre-existing real operator tmux sessions, confirmed untouched before and after install; a throwaway password-protected session was created, verified (masked input, no plaintext leak), and cleanly deleted as live evidence.

[v1.0.33] — 2026-07-02

Fixed

- **Ubuntu/dash portability: `obtain_local_deps.sh` failed `sh -n`** (test 74, §11.4.67), blocking setup.sh's PATH-export on hosts where `/bin/sh` is dash (Ubuntu). The `userns` bash array (podman `--userns=keep-id`) was the sole non-POSIX construct; replaced with a plain string + unquoted expansion — identical under bash, now parses under dash. Root-caused live on thinker/amber (tmux 3.6a compiled but the verification gate refused PATH-export).

[v1.0.32] — 2026-07-02

Changed

- **Fully-elastic ("liquid") per-session memory (Constitution §105/§106).** tmx scopes now launch with `MemoryMax=infinity` + a `MemoryMin=128M` floor instead of a hard `MemoryMax` cap — a session uses all available RAM/zram and is NEVER per-scope OOM-killed. Genuine exhaustion is handled system-wide by systemd-oomd + zram + the user-slice backstop (OOM-Protect). `TMX_MEM` becomes an OPT-IN *soft* `MemoryHigh` throttle (reclaim, never kill); `TMX_MEM=auto` uses the host-adaptive size. Fixes ">3 concurrent agents get OOM-killed" under memory pressure. Applied to both tracked sources (`tmx.template`, `tmx.safe`); test 15 updated (memory.max=max, memory.min=128M, TMX_MEM=3G→memory.high=3G) — verified 8/8 live.

[v1.0.31] — 2026-06-29

Fully-autonomous ROOT-FREE build + 4-host install/retest hardening. A live install+retest on all four hosts (nezha, amber, thinker, mistborn) caught six findings — every one a test-harness / build-orchestration robustness issue, NOT a product defect (tmx builds, sessions, and the root-free build work on every host) — all fixed and re-verified GREEN on all four hosts: Linux x86_64 ×3 (including a host with no system jemalloc) and macOS arm64.

Added

- **Root-free, sudo-free, interaction-free build path.** `obtain_local_deps.sh` obtains a prebuilt Zig `cc` toolchain (clang+lld+libc+crt, sha256-verified) into git-ignored `.local-`

deps/, and build_native.sh builds tmux from the 3.6a release tarball with it — so setup.sh produces a working tmx even on a host with no C toolchain, no root, no sudo, and no interactive prompts. Proven on nezha (test 71 12/0). Enforced by the new CM-NO-SUDO-NO-INTERACTION verify gate (rejects any sudo/su command execution and any interactive prompt anywhere in the automation/test scripts).

Fixed

- **test 68 (session lifecycle) — §11.4.1 harness FAIL-bluff.** pty_harness.sh + 68_session_lifecycle.sh accepted TERM=dumb over non-interactive SSH, so the tmux client never attached and the lifecycle test failed while the product worked. Now rejects dumb/non-cursor-addressable TERM via a tput -T <cand> clear probe and falls through to a usable terminal; + macOS /tmp→/private/tmp dir-canonical fix. RED→GREEN N=3 (78/0 under forced TERM=dumb).
- **test 71 — unbounded download hang.** The Zig download had no timeout, so a throttled mirror hung the test ~8h. Now bounded (curl --max-time / --speed-limit) → fails fast (exit 28 in 30s) → honest SKIP when the mirror is throttled; the normal path is unaffected (12/0).
- **amber build-orchestration (native host-cc path on a constrained host).** (a) build_native.sh exports LD_LIBRARY_PATH around ./configure so the autoconf run-test can load the obtained local jemalloc (was EXIT 77 "cannot run C compiled programs"); (b) setup.sh's containerized-success check applies resolved.env LD_LIBRARY_PATH so a working containerized build is no longer false-failed into a needless native fallback; (c) the native build obtains a LOCAL static libtinfo.a (local ncurses --with-termlib, or a host libtinfo.a fallback) so the binary links no dynamic -ltinfo → CM-NO-DYNAMIC-LIBTINFO gate + test 61 pass (readelf: 0 libtinfo DT_NEEDED); (d) setup.sh auto-obtains the Go toolchain → CM-TMX-STATE-GO-PRESENT gate pass.
- **macOS self-containment.** install_deps.sh honors INSTALL_DEPS_FORCE_DISTRO before OS-detection (test 70 C4/C5); run_all.sh adds the brew gnubin PATH (GNU timeout) and re-execs under bash ≥ 4 (tests 30/39/49/57) — a no-op on Linux.

Changed

- **test 70 C8/G1 + C10 reconciled (§11.4.120)** to assert the new _cb_ok containerized-fallback wiring (teeth proven by the paired §1.1 mutation — neither fake-passed nor reverted).

Verified — all 4 hosts GREEN on this release (captured evidence under `qa-results/loop-20260629/`, no bluff)

Host	OS	`run_all`	`tmx` → tmux 3.6a
nezha	Linux x86_64	PASS=58 FAIL=0 SKIP=12 (root-free test 71 `12/0`)	✓
amber	Linux x86_64 (no system jemalloc)	PASS=50 FAIL=0 SKIP=20	✓
thinker	Linux x86_64	PASS=50 FAIL=0 SKIP=20	✓
mistborn	macOS arm64	PASS=61 FAIL=0 SKIP=9	✓

[v1.0.30] — 2026-06-29

Workable-item key prefix corrected ATM- → TMX- (this project, not ATMOSphere). Cross-platform install hardening: curl one-liner installer, libevent/ncurses local-dependency obtain, native-build fallback for rootless-Podman hosts, and an escaped-colon session-color fix.

Changed

- **Workable-item ticket key migrated `ATM-` → `TMX-` everywhere (§11.4.54 / §11.4.35).** This project was migrated from ATMOSphere and wrongly inherited ATMOSphere's `ATM-` key prefix; the correct key for this project is `TMX-`. Migrated across the Go workable-items tooling (added `const TicketPrefix="TMX-"` / `TicketLabel="TMX-ID"`, routed all literals through them — fix-at-source §11.4.1), the tracked SQLite DB values (`items.atm_id` + child-table refs via a focused value-only `REPLACE` migration with a §9.2 backup — never a renumbering rebuild, per §11.4.54), golden testdata, and all trackers (Issues/Fixed/CONTINUATION/CHANGELOG + docs/workable-items + exports). The Go symbol `ATMID`, the DB column name `atm_id`, and the universal `schema.sql` are intentionally kept (constitution-synced schema, §11.4.28 — values migrated, schema not forked). Legitimate `ATMOSphere` origin-history references are preserved (the rename targets the hyphenated `ATM-` token only). 56 items → TMX-001..TMX-056. §11.4.93 byte-

identical round-trip + validation (0 findings) stay GREEN; DB `foreign_key_check` + `integrity_check` clean.

Added

- **curl-obtainable installer (`scripts/install.sh`).** `curl -fsSL .../scripts/install.sh | bash` → clone-recursive (+ submodules) → `setup.sh` (build + verify + gated PATH export) → `run_all.sh` → confirm the `.bashrc`/`.zshrc` snippet + `tmx` resolvability. §9.2 refuse-to-clobber a foreign directory; idempotent re-run (update mode); rc-detection by file-existence; env overrides (`TMX_INSTALL_DIR`/`TMX_REPO_URL`/`TMX_INSTALL_BRANCH`/`TMX_INSTALL_NO_SETUP`). README one-liner + `docs/scripts/install.md`. Test 69 PASS=9/0/0 (real offline recursive-clone proof).
- **libevent + ncurses local-dependency obtain (completes the §11.4.77 local-deps mechanism).** `obtain_local_deps.sh` now resolves-or-obtains libevent 2.1.12 + ncurses 6.5 (sha256-verified from authoritative sources) into git-ignored `.local-deps/`, so a forced native build works on a minimal host lacking the `-dev` packages. `build_native.sh` consumes them via `-I/-L` + `PKG_CONFIG_PATH`. Acid-test: a real `tmux ./configure` finds the obtained local libevent (exit 0; control without it → `libevent not found`).
- **Native-build fallback for rootless-Podman hosts (`setup.sh`).** When the containerized build fails (e.g. rootless Podman `/etc/subuid+/etc/subgid` exhaustion → `lchown /etc/gshadow: invalid argument`), setup falls back to a native host build (§11.4.101) and emits the `usermod --add-subuids/--add-subgids` + `podman system migrate` repair hint. Both builds failing surfaces both errors + exits non-zero (no silent green). Success path unchanged.

Fixed

- **Escaped-colon session names lost their color on Linux (`scripts/tmx.template`).** The Linux dispatch passed the RAW `-s` value to tmux while the socket label / has-session check / color application keyed off the SANITISED name → `tmx new -s 'name\:x:color'` created a mis-named session, has-session failed, and the color was never applied (status-style stayed tmux-default green). Now passes `$NAME` to `-s` (parity with the already-correct Darwin path). Test 63 → PASS=8/0/0.

Verification

- nezha (Linux x86_64): TMX-001 GATE A go test GREEN; GATE B validate 0-findings + zero-ATM-keys (non-export) + ATMOSphere keep-list intact + DB foreign_key_check/

integrity_check clean; test 63 8/0/0, test 67 5/0/0, test 69 9/0/0.
- v1.0.30 readiness audit: GO — no release blocker.
- libevent obtain acid-test: real `tmux ./configure` exit 0 against the obtained local libevent (control: exit 1).

Non-blocking follow-ups tracked: TMX-068 (obtain Go for amber → tmx-state build), TMX-070 (document the rootless-podman subuid fix + install HTTPS-rewrite edge), DB↔Fixed.md SSoT drift (pre-existing, orthogonal), test-coverage gaps G1–G5.

Co-Authored-By: Claude Opus 4.8 (1M context) noreply@anthropic.com

---

[v1.0.29] — 2026-06-28

Full session lifecycle (name:color + password + cwd + detach/rejoin + recycle + delete-reset) proven working with real live tmux sessions. Cross-platform local-dependency mechanism. Six cross-platform defects found and fixed via §11.4.108 clean-target host validation.

Added

- **`tmx delete -t NAME` (clause 7 — delete-to-reset).** Kills the session + clears persisted state (dir/color/password) via `tmx-state-bin forget`. Re-create yields default color, `$HOME` dir, and a fresh password prompt. (`scripts/tmx.template`, `scripts/tmx-state/main.go`)
- **Idle-timeout session recycler (clause 6 — `tmx-recycler.sh`).** External poll-watcher launched by `tmx new` when `TMX_RECYCLE_IDLE_SECS > 0` (default 900s). Atomic `if-shell` race-guard ensures an attached session is never killed. Records state before teardown (tmux #1174 mitigation). `=0` disables. Linux: scope-stop + kill-session; macOS: kill-session only. (`scripts/tmx-recycler.sh`, `scripts/tmx.template`)
- **`cmdRecord` preserves Color+PasswordHash across detach (pre-existing bug fix).** `cmdRecord` was rebuilding the Session struct as a literal, silently dropping Color and PasswordHash on every detach/recycle. Now starts from the existing record and updates only cwd fields. (`scripts/tmx-state/main.go`, `scripts/tmx-state/main_test.go`)
- **7-clause full-automation PTY lifecycle test (`68_session_lifecycle.sh`).** Drives real live tmux sessions via `tmx send-keys` PTY harness: create+color+password, cwd capture, kill-HUP detach survival, rejoin (same dir+color+password re-prompted), idle-recycle (state remembered), delete-reset. PASS=78/0/0 ×3 on nezha. (`scripts/tests/68_session_lifecycle.sh`, `scripts/tests/lib/pty_harness.sh`)

- **Darwin Homebrew absolute-path resolution in `setup.sh`.** Resolves brew by absolute path (`/opt/homebrew/bin/brew`, `/usr/local/bin/brew`) + prepends to PATH before the hard-exit gate. Fixes mistborn (macOS) `setup.sh` exit 3 under non-interactive SSH PATH lacking `/opt/homebrew/bin`. (`scripts/setup.sh`)
- **Test 67 C3 prefix-agnostic probe + LD_LIBRARY_PATH proof.** C3 now detects the resolved SO's exported symbol prefix via `nm` (`mallctl` vs `je_mallctl`) and compiles the probe accordingly. LD_LIBRARY_PATH assertion (the real product runtime mechanism) replaces the unreliable ldd path-assertion. Honest macOS local-build-override limitation documented. (`scripts/tests/67_local_deps.sh`)
- **Wrapper terminfo fallback + create-result check.** Falls back to a terminfo-present TERM when `infocmp "$TERM"` fails. Checks `has-session` after create and fails loud with the tmux error instead of silently `exec attach`-ing onto a dead server. (`scripts/tmx.template`, `scripts/tests/lib/pty_harness.sh`)

Fixed

- **test 17 T4.2 scrollback retention reconciled to race-free `#{history_size}`.** v1.0.27 had regressed T4.2 to a grid `capture-pane` line count; restored the atomic-counter design (A49/v1.0.25). (`scripts/tests/17_scrollback_copy_mode.sh`, `scripts/tests/meta_test_false_positive_proof.sh`)
- **codegraph tests 20/21/22 SKIP-with-reason when CLI absent.** (`scripts/tests/20_codegraph_installed.sh`, `21_codegraph_index_present.sh`, `22_codegraph_mcp_wired.sh`)
- **test 27 macOS stale-cd race.** Replaced blind `sleep 0.4` with poll on `#{pane_current_path}`. (`scripts/tests/27_state_persistence.sh`)
- **test 60 T3 socket-path guard.** Realpath-projected measurement + `/tmp` fallback. (`scripts/tests/60_wrapper_tmux_bin_valid.sh`)
- **containerized build Docker-compatible.** Gated `--userns=keep-id` to podman only. (`scripts/build_containerized.sh`)
- **cwd-persist hook resolves project binary (not bare `tmx`).** (`scripts/tmx-shell-init.sh.template`, `scripts/setup.sh`)

Verification

Captured physical evidence (real, not fabricated):

- **nezha (Linux x86_64):** test 68 PASS=78/0/0 ×3, test 67 PASS=5/0/0 ×3, run_all PASS=52/0/13, go test GREEN, §1.1 meta-sweep 58 caught / 0 escaped.
- **thinker (Linux x86_64, tmux 3.4):** setup exit 0, run_all 47/0/19, cwd-persist test 27 PASS 3/3 (was DEAD pre-fix).
- **mistborn (macOS arm64, bash 3.2.57):** brew-abs-path resolved off-PATH, tmux 3.6a runs, tests 27+60 RED→GREEN.

- **amber (Linux x86_64, Docker 29.4.1):** obtain-jemalloc acid-test GREEN (binary runs via LD_LIBRARY_PATH, was exit 127).

Co-Authored-By: Claude Opus 4.8 (1M context) noreply@anthropic.com

---

[v1.0.28] — 2026-06-28

Per-session password protection — any session can be optionally password-gated. Passwords stored as SHA-256 hashes, verified on attach.

Added

- **Per-session password protection (`tmx new -s NAME` prompts for optional password).** After creating a new interactive session, the operator is optionally prompted for a password. If set, every subsequent `tmx attach -t NAME` requires the correct password before the session can be accessed. Passwords are stored as hex-encoded SHA-256 hashes in `~/.tmx/state.json` (schema v2→v3, additive `password_hash` field; forward+backward compatible). `tmx-state-bin` gains `set-password` / `verify-password` subcommands (v1.2.0).
- **State schema v3** — additive `password_hash` field on `Session`. A schema-2 file (no password field) loads with `PasswordHash==""` (no guard); a schema-3 file read by an old binary ignores the unknown field. No migration script needed.
- **Test 66 (`66_session_password.sh`)** — 5 invariants × 3 iterations: set-password stores hash (exit 0), verify-password accepts correct (exit 0), rejects wrong (exit 1), accepts any when no password set (exit 0), empty password clears protection (exit 0).
- **Meta-test mutation M27** — makes `verifyPassword` always return true → test 66 T3 (wrong password → exit 1) FAILs. Paired §1.1 mutation proves the password guard is not a bluff.
- **Governance sync** — constitution submodule advanced `1d408cb` → `1576d3d`; §11.4.159–§11.4.170 propagated to CLAUDE.md, AGENTS.md, QWEN.md, GEMINI.md, Constitution.md.

Security

- **No plaintext passwords** — passwords stored as SHA-256 hashes only. The `set-password` command hashes before writing; `verify-password` hashes the input and compares. Empty password clears the hash entirely.

- **Graceful degradation** — if `tmx-state-bin` is absent or fails, the wrapper continues without password protection (tmx-never-breaks invariant). The password feature is opt-in only.

[v1.0.27] — 2026-06-19

Anti-bluff completeness burn-down: M24 test-coverage gap closed, A2 run_all OS-aware, D2 /tmp scratch-SKIP, A51 operator-escape name:color prompt rejection fixed.

Fixed

- **A51 — `tmx-shell-init` prompt rejects `:` in session names (operator-escape, §11.4.138).** The interactive login prompt validated against `[A-Za-z0-9_.-]{1,64}`, blocking `name:color` input. Now allows `:` and `#` via `[A-Za-z0-9_#.:-]{1,80}`. The `tmx` wrapper's existing `_parse_session_value()` already parsed the color; only the prompt layer was stale. Fixed in `tmx-shell-init.sh.template` + `tmx-ssh-dispatch.sh.template`. Test 65 (operator-escape guard) PASS=6/6. Root cause per §11.4.108 SOURCE→USER-VISIBLE gap (GREEN suite, broken-for-user feature). ([7aefdf2..current])
- **M24-ESCAPE-001 — meta-test M24 mutation now CAUGHT** (was escaping since v1.0.9 `f151d13`). Root cause: `re.subn(pat, '', src, count=1)` removed from `_apply_color()` only, while test 26 exercises `_apply_host_color()`. Removed `count=1` → strips from BOTH functions → M24 CAUGHT. Meta-test: 37 CAUGHT / 0 ESCAPED / 17 SKIPPED.
- **A2 — `run_all.sh` OS-aware binary resolution** on native macOS (`tmux/build-darwin` first, fallback to `tmux/build`).
- **D2 — All test scratch paths route through `${TMPDIR:-/tmp}`** with writability preflight guard (§11.4.3 SKIP-with-reason), converting false-FAIL under host disk-pressure to honest SKIP. 35 test files updated.
- **Test 35 `LONG65→LONG81`** reconciled with the 80-char max-length bump per name:color feature (§11.4.120).

[v1.0.26] — 2026-06-19

Per-session color via `tmx new -s name:color` — the first operator-chosen per-session color. Every session was previously the single hostname-derived color; now each session can carry its own validated, persisted color.

Added

- **Per-session color (`name:color[:ignored]`)** — an operator can choose an explicit tmux color by typing it into the `-s` value,

colon-delimited. `tmx new -s work:red` paints all four "green" surfaces (status bar bg, active-pane border, clock-mode-colour, current-window marker) red. Accepted formats (validated): tmux names (`red`/`green`/…/`colour255`, case-insensitive) and `#RGB`/`#RRGGBB` hex (true-color). An invalid color is rejected before any session/socket is created (§11.4.6 — no bluff).
- **Persistence + precedence** — the color is stored in `~/.tmx/state.json` (schema 1→2, additive `color` field; forward+backward compatible) and re-used on later bare-name runs. Precedence: inline > persisted > hostname-derived > default-green. `tmx-state-bin` gains `set-color` / `get-color` subcommands + a `COLOR` column in `list` (v1.1.0).
- **Escaped colon in name** — `tmx new -s 'a\:b:cyan'` produces name `a:b` (sanitised `a_b`) with color `cyan`. Extra `:fields` after the color are ignored (forward-compatible).
- **Cross-OS parity (§11.4.81)** — works identically on Linux + macOS; the macOS bridge forwards the `-s` value verbatim, the VM-side wrapper parses.

Changed

- `tmx-state-bin` `v1.0.9` → `v1.1.0` (additive: `Session.Color`, `set-color`/`get-color`, `list` 4th column). Schema version 1→2 (additive, no migration).
- `tmx.template` sources new `scripts/tmx-color-lib.sh` (pure helpers `_parse_session_value` / `_color_valid`); resolves `EFFECTIVE_COLOR` and applies via new `_apply_color` (mirrors `_apply_host_color`).

Tested (four-layer §11.4.4(b) + §102 operator-path)

- `63_session_color.sh` — **8/8 PASS** reading live `show-options` from the real server (T1 name, T2 #hex, T3 all-4-surfaces, T4 persistence, T5 invalid-rejected, T6 escaped-colon, T7 extra-ignored, T8 hostname fallback). Deterministic 3× (§11.4.50).
- `64_session_color_parse_unit.sh` — **17/17** pure parser/validation unit tests, incl. bash↔Go canonical-name-list parity.
- §1.1 paired mutations **M25** (strip `_apply_color`) + **M26** (neutralize `set-color`) both **CAUGHT** by test 63 — the tests genuinely catch the defect class.
- HelixQA Challenge **TMUX-CH-53** (blocker).
- Full-suite `run_all.sh`: PASS=55, the 2 FAILs (test 13 Darwin `ulimit -u` honest-gap §11.4.81; test 58 pre-existing mouse-drag attach-refresh) are **pre-existing + environment-specific, not regressions** — proven by empty diff overlap with mouse/bind/copy surfaces.

Known / tracked separately

- `M24-ESCAPE-001` (pre-existing since v1.0.9, `f151d13`): the hostname 4-surface mutation escapes test 26. Tracked OPEN in `Issues.md`, fixed separately — not a regression of this release.

[v1.0.25] — 2026-06-16

Governance + tooling currency: advance the HelixConstitution submodule to its latest upstream (`1d408cb`), fix host-level codegraph version skew, and clean dead test-socket cruft — per the operator mandate "we MUST ALWAYS target the latest versions of all our submodules." No tmux product-surface change.

Changed

- **HelixConstitution submodule advanced `3f4d690` → `1d408cb`** (+67 upstream commits, new universal anchors §11.4.140–158). Functional inheritance verified — `18_constitution_inheritance.sh` **PASS=10/0** (the project's `@constitution/CLAUDE.md` import + INHERITED-FROM blocks resolve to the new constitution). The release gate is unaffected (`verify.sh` enforces the §11.4.87..99 propagation set, already mirrored).
- **Containers submodule** confirmed at latest (`18ed03d`).

Fixed (host-level — no repo change)

- **codegraph version skew on macOS** — `codegraph --version` reported `0.9.9` while npm had updated `0.9.9 → 1.0.1`: a stale `~/.local/bin/codegraph` symlink (→ `~/.codegraph/ versions/ v0.9.9`) shadowed the npm-managed `1.0.1`. Repointed the symlink → both hosts now resolve **1.0.1** (matches npm-latest). Closes the §11.4.80/§107 "npm exit 0 was bluffing" entry the cadence script had caught.
- **macOS host hygiene** — removed **668 dead tmux test sockets** (`/tmp/tmux-501`, 678 → 10) without touching any live/ real session (not a `kill-server`).

Fixed (tests — swap-aware OOM, §11.4.81/§11.4.50)

- **Tests 12 & 14 (destructive cgroup OOM) failed deterministically on nezha** because the host gained **15 GiB swap**: a cgroup at its `MemoryMax` enforces the cap by reclaim/ **swap** (memory.current pins at the cap, `oom_kill` stays 0)

instead of OOM-killing, so no OOM fired. Root-caused via the cgroup's own `memory.events` counter (FACT: `oom_kill` 0 with swap, **295** with swap disabled). Fix: the test scope now sets **`memory.swap.max=0`** (test 12 via `systemd-run -p MemorySwapMax=0`; test 14 by writing the delegated cgroup file) so exceeding the cap OOM-kills deterministically on swap-enabled AND swapless hosts — touching only the test scope, never host-wide swap. Now PASS 3/3.

Validation (dual-host release gate)

- **nezha** `verify.sh` (TMX_TEST_DESTRUCTIVE=1) GREEN + meta-test 52 CAUGHT / 0 ESCAPED; tests 12 & 14 PASS 3/3 with real OOM-kill evidence after the swap-aware fix.
- **macOS** `run_all` GREEN.
- `setup.sh` re-run + verified on both hosts.

Known follow-up (non-release-blocking)

- Verbatim/cascade propagation of the new §11.4.140–158 anchors into the project's `CLAUDE.md/AGENTS.md/QWEN.md` verbatim-mirror sections (for non-`@import` agents) is tracked separately — functional inheritance already works via the `@import`, and the release gate does not enforce these anchors.
- Operator-pending on nezha: `sudo bash scripts/build_oom_set.sh --install` + `sudo apt-get install -y stress-ng` to promote tests 08/12/14 from honest SKIP to PASS.

Sources verified 2026-06-16

- https://github.com/HelixDevelopment/HelixConstitution
- https://docs.npmjs.com/cli/v10/commands/npm-install

[v1.0.24] — 2026-06-16

On-demand container-distribution orchestrator: a real binary (`scripts/tmx-orchestrator/`) that drives the `vasic-digital/containers` submodule to schedule, deploy, health-check, and tear down containers on remote test hosts (nezha.local) — for heavy testing that depends on real services running.

Operator mandate: "Make a proper binary using the Containers submodule lib to orchestrate the distribution — for when/if we need it for our testing needs." The tmux product surface (the

hardened tmux 3.6a binary + `tmx` wrapper) is **unchanged** from v1.0.23; this release adds testing tooling + extends + hardens the consumed Containers library.

Added

- **`scripts/tmx-orchestrator/`** — a Go consumer of the decoupled `digital.vasic.containers` submodule (via `replace`, CONST-051 — no tmux-specifics added to the submodule). Subcommands: `hosts` (register + SSH-probe remote hosts → real `/proc` CPU/MEM/cores), `distribute` (schedule + `podman run -d -p host:container` a real container on the best remote host + poll-until-ready health check), `down` (teardown, exits non-zero when teardown can't be confirmed). nezha.local registered in `Containers/.env` (gitignored). Binary `scripts/tmx-orchestrator-bin` gitignored, regenerated via `go build` (§11.4.77).
- **`scripts/tests/62_distribution_orchestrator.sh`** — opt-in (`TMX_TEST_REMOTE=1`) end-to-end test: builds, probes nezha, deploys a real nginx container + asserts HTTP 200, tears down clean. SKIP-with-reason by default.

Fixed (Containers library — extended/hardened upstream per §11.4.76)

- **Port publishing** in the remote deploy path (`ContainerRequirements.Ports` + `-p` rendering) so a distributed service is reachable for health checks (Containers `1b9da9b`).
- **Command-injection** in the port-publish protocol — allowlisted (Containers `82bd586`).
- **ControlMaster socket leak** — `SSHExecutor.Execute` released the pooled SSH connection ref only on success; now releases on all paths (`a856865`, RED→GREEN regression test on real nezha) + the same-class `ExecuteStream` early-error leak (`18ed03d`).
- **`tmx-orchestrator down` FAIL-bluff** — reported "teardown complete" + exit 0 even on an unreachable host; now exits non-zero when teardown can't be confirmed.

Proven evidence (real captured runtime, no guessing per §11.4.6)

Captured under `docs/qa/2026-06-16-tmx-orchestrator/`:

Proof	Result
`tmx-orchestrator distribute` (macOS → nezha over SSH+podman)	real nginx `0.0.0.0:18080->80/tcp`, **HTTP health 200** (podman ps + curl confirm)
`down`	

Proof	Result
	container **REMOVED-CLEAN**; unreachable host → **exit 1**
pool-leak regression (`pkg/remote/execute_release_test.go`)	RED refs=1 → GREEN refs=0 on nezha
test 62 (`TMX_TEST_REMOTE=1`)	PASS=4/0
Containers full suite on nezha (unit + `-tags=integration` vs real podman)	all 40 packages `ok`
Adversarial code-review (§11.4.125/§11.4.134)	2 major + 4 minor found → all fixed → re-review **GO**
macOS `run_all`	PASS=55 FAIL=0 SKIP=6 (zero regression)

Notes

- Submodule pointer `vasic-digital/containers` advanced 61e01dc → 18ed03d (3 fix commits pushed to github + gitlab).
- `Issues.md` zero open; closure recorded as `Fixed.md` A48.

[v1.0.23] — 2026-06-16

Cross-distro Linux hardening: the tmux binary no longer emits the `/lib64/libtinfo.so.6: no version information available` warning on every invocation, plus three test-determinism fixes — validated dual-host on macOS (Darwin arm64) and nezha (ALT Linux x86_64).

The Linux build of tmux is compiled in an `ubuntu:22.04` container whose `libtinfo` carries versioned symbol nodes (`NCURSES6_TINFO_5.0.*` / `5.8.*`). When that binary ran on the `nezha.local` host (ALT Linux), whose `libtinfo.so.6` has **zero** NCURSES version nodes, the dynamic loader printed `/lib64/libtinfo.so.6: no version information available` on **every** tmux invocation — polluting the stderr of every command. This release links terminfo **statically** so the warning is structurally impossible on any distribution, and fixes three test-harness timing races discovered while re-validating the full suite on real Linux hardware.

Fixed

- **libtinfo cross-distro warning eliminated (cross-distro portability, §11.4.81).** `docker/build_inside_container.sh` now passes `LIBTINFO_LIBS="-l:libtinfo.a"` to `./configure`, linking

terminfo from `ubuntu:22.04`'s static archive instead of `libtinfo.so.6`. The ELF carries **no** `libtinfo.so` `DT_NEEDED` entry and **zero** `NCURSES6_TINFO` versioned refs, so the warning cannot fire on any host. This is **partial**-static (terminfo only) — `glibc`, `jemalloc`, and `libevent` stay **dynamic** (`DT_NEEDED` still lists `libjemalloc.so.2`), preserving the `LD_PRELOAD`-jemalloc architecture and the build-time `-ljemalloc` linkage. NOT `--enable-static` (which produces a fully-static binary that drops jemalloc and breaks `LD_PRELOAD`). Static terminfo still reads the host's `/usr/share/terminfo` at runtime — terminfo data is never bundled.
- **Test 27 (`tmx-state` cwd persistence) — deterministic FAIL on macOS fixed (§11.4.1 source-layer).** Phase 3 recorded `#{pane_current_path}` after a blind `sleep 0.4`, but macOS tmux updates the pane cwd ~1.5 s after a `send-keys "cd …"` (measured) — so it captured the stale path and FAILed 3/3. Now polls until the cd is reflected before recording. PASS 3/3 on both macOS and Linux.
- **Tests 12 & 14 (cgroup memory-pressure / concurrent-OOM independence) — flaky FALSE-NEGATIVE fixed (§11.4.1 / §11.4.50).** The kernel OOM-kill fires asynchronously and, on a `kernel.dmesg_restrict=1` host, `journalctl -k` ingests the line with lag; a fixed-`sleep` single sample raced it. Now polls the kernel ring for the OOM line up to a generous timeout (assertion unchanged). PASS 5/5 deterministic on nezha with real OOM-kill evidence.

Added

- **`scripts/tests/61_no_libtinfo_version_warning.sh`** — cross-platform regression guard (§11.4.81): T1 (both OSes) the binary emits no dynamic-loader version warning; T2 (Linux) `ldd` shows no `libtinfo.so` and `objdump` shows 0 `NCURSES6_TINFO` refs; T3 (Linux) `DT_NEEDED` still lists `libjemalloc.so` (fix did not regress jemalloc). Darwin SKIPs T2/T3 with reason; T1 is the shared positive proof.
- **`scripts/verify.sh` gate `CM-NO-DYNAMIC-LIBTINFO`** — pre-build layer-1 guard: refuses to bless a Linux binary that dynamically links `libtinfo.so`. Teeth proven (the distro `/usr/bin/tmux`, which links `libtinfo.so.6`, trips it); Darwin PASSes by non-applicability.

Proven evidence (real captured runtime, no guessing per §11.4.6)

Captured under `docs/qa/2026-06-16-libtinfo-crossdistro/`:

Host	Proof
nezha (ALT	rebuilt binary `ldd` → 0 `libtinfo` deps; `tmux -V` stderr → **0** warning lines (was 1/invocation); test 61 PASS

Host	Proof
Linux x86_64)	3/0/0; full `verify.sh` (TMX_TEST_DESTRUCTIVE=1) **EXIT 0, PASS=49 FAIL=0 SKIP=11**, `CM-NO-DYNAMIC-LIBTINFO` PASS; tests 12 & 14 PASS 5/5 with kernel OOM-kill detected + user.slice survived; meta-test **52 CAUGHT / 0 ESCAPED**
Mistborn (Darwin arm64)	full suite (isolated TMPDIR, Mach-O binary) **EXIT 0, PASS=55 FAIL=0 SKIP=5**; test 27 PASS 3/3; test 61 T1 PASS (Darwin)

Notes

- The destructive cgroup-isolation tests (`12`/`14`) require `TMX_TEST_DESTRUCTIVE=1`
  - `stress-ng`; they remain SKIP by default and run only on a dedicated Linux test host. `08_oom_score_adj` stays an honest SKIP (needs root / setcap helper).
- No change to the macOS (Mach-O) build or the shipped `tmx` wrapper behaviour — the libtinfo fix is confined to the containerized Linux ELF link.

Sources verified 2026-06-16

Static-terminfo linking + symbol-versioning + runtime terminfo resolution cross-referenced against latest upstream/authoritative sources (§11.4.99):

- https://mhcerri.github.io/posts/building-tmux-statically/
- https://github.com/tmux/tmux/blob/master/configure.ac
- https://www.man7.org/linux/man-pages/man5/terminfo.5.html
- https://invisible-island.net/ncurses/ncurses-mapsyms.html
- https://invisible-island.net/ncurses/ncurses.faq.html

[v1.0.22] — 2026-06-13

Apple `container` integration: on-demand containerized Linux under macOS for testing — build tmux 3.6a inside a real Linux VM and run the suite, no remote Linux host required.

The project is native dual-OS (Linux + macOS), but on a macOS workstation there was previously no host-local way to exercise the **Linux** build of tmux — Linux validation depended on the remote `nezha` host. This release adds on-demand containerized Linux under macOS using Apple's native `container` runtime (1.0.0), so a

developer on Apple Silicon can build the Linux ELF tmux binary inside a real Linux VM and run the project's own test suite against it with one command.

Added

- **Containers-submodule backend (extended, not reimplemented — §11.4.76).** The Apple `container` 1.0.0 runtime was incorporated into the `vasic-digital/Containers` submodule as a new generic `pkg/crossbuild/apple_container.go` backend exposing `RunInLinuxContainer`, with unit + integration tests and a challenge. Proven on this macOS 15.5 / arm64 host: a real `container run` returns `Linux aarch64` (while the host is Darwin), a host-directory mount round-trips, and a paired mutation that strips the `--mount` flag exits 99. The cross-build capability lives in the reusable submodule per §11.4.74 (catalogue-first) — the project consumes it, it is not duplicated in-tree.
- **tmx-side harness `scripts/test_apple_container.sh`.** Builds tmux 3.6a INSIDE an Apple-`container` Linux VM (genuine Linux build: `osdep-linux.o` + `libjemalloc.so.2` + `libevent_core`) and runs `run_all.sh` against that Linux binary, capturing real PASS/FAIL/SKIP evidence. Stops + removes the transient container on every exit path. Honest exit codes: `0` PASS, `1` real FAIL, `2` build defect, `3` SKIP (runtime/kernel absent or not macOS).

Proven evidence (real captured runtime, no guessing per §11.4.6)

Captured under `docs/qa/2026-06-13-apple-container/linux-run/`:

Artifact	Proof
`uname.txt`	in-container `uname -s -m` = **`Linux aarch64`** (host is Darwin/arm64)
`tmux-version.txt`	built binary reports **`tmux 3.6a`**
`elf-proof.txt`	ELF magic `7f 45 4c 46` + `ldd` shows `libjemalloc.so.2`, `libtinfo`, `libevent_core`, `libc` — a real Linux ELF, jemalloc-linked
`build.log`	full in-VM configure+compile transcript (Linux gcc, `osdep-linux.o`)
`run_all.log` / `summary.txt`	suite result **PASS=30 / FAIL=0 / SKIP=28** against the Linux binary

Host: Darwin arm64. Container OS: Linux aarch64. Base image: `docker.io/library/ubuntu:22.04`.

Honest topology SKIPs (§11.4.3 / §11.4.81 — not pass, not fail)

The 28 SKIPs are honest topology gaps, never silent passes: the minimal container VM has no user systemd session, so cgroup/scope tests SKIP-with- reason (`08_oom_score_adj`, `09_crash_isolation_scope`, `12_memory_pressure_under_cap`, `13_tasksmax_stress`, `14_concurrent_oom_independence`, `15_per_session_cgroup_distinct`, `24_cpu_cap_enforcement`); physical-terminal / real-clipboard / real-mouse tests SKIP (no DISPLAY/PTY tty in the VM — `44–48`, `55–59`); and host-tooling / cross-host tests SKIP (`16_window_name_strips_exe`, `18_constitution_inheritance`, `20–22` CodeGraph, `32_ssh_dispatch_remote_nezha`, `39_state_unwritable`, `40_macos_linux_parity`, `41_docs_user_guides_render`, `43_e2e_cwd_persist_real_shell`, `51_workable_items_db_integrity`). The 30 core tmux tests (smoke, session lifecycle, jemalloc-mapped proof, history-limit, clear-history, scrollback, hostname-colour, …) RAN and PASSED.

Notes

- No tmux source or wrapper behaviour changed in this release — this is a testing-capability addition (the Linux-under-macOS harness + the Containers-submodule backend it consumes). The shipped tmux 3.6a binary and `tmx` wrapper are byte-for-byte the v1.0.21 product.

Sources verified 2026-06-13

Apple `container` documentation cross-referenced before publishing the harness usage + requirements (per §11.4.99):

- Apple `container` documentation — https://apple.github.io/container/documentation/

[v1.0.21] — 2026-06-13

Copy/paste root-cause fix — the terminal owns the mouse by default (native multi-line select + right-click→Copy + scroll work everywhere); tmux mouse on demand via `prefix m`.

Operator reports (2026-05-28 .. 2026-06-13, across iTerm2 / Terminal.app / a Linux terminal / WezTerm): "copy paste by selecting multiple lines of code does not work properly. It must work on Linux and macOS. I must scroll and always be selectable for copying — right-click → Copy." Three prior binding-level fixes (v1.0.15 / v1.0.18 / v1.0.20) added ever-more tmux mouse

bindings yet the operator still could not reliably select/copy — the signal that the *architecture*, not the bindings, was wrong (systematic-debugging: 3+ fixes failed → question the architecture).

Fixed

- **Root cause (proven at the wire level, no guessing per §11.4.6).** With `set -g mouse on`, on attach tmux emits mouse-tracking DECSET *enables* (`CSI ?1000h ?1002h ?1006h`) to the outer terminal, putting it into mouse-reporting mode — which **suppresses the terminal's own native selection and right-click→Copy**. No tmux binding can intercept a terminal's right-click→Copy menu, so the only way select/copy "always" works is to let the terminal own the mouse. **Captured proof:** a PTY attach-stream capture shows `mouse on` emits **6** mouse-enable DECSET; `mouse off` emits **0**.
- **The fix.** `scripts/tmux.conf.template` default flipped `set -g mouse on` → `set -g mouse off`. The terminal now owns the mouse: native click-drag selection (incl. **multi-line**), **right-click→Copy**, and native scroll all work identically on Linux and macOS, on every emulator. The complete tmux mouse stack (wheel-scrollback inside TUIs + drag-select-to-OS-clipboard) remains available **on demand** via the `prefix m` toggle — which flips `mouse on`, at which point tmux *does* emit the enables (proven: 3 DECSET enables appear after `prefix m`). `prefix P` keyboard paste and native paste (Cmd-V / right-click→Paste) are unaffected.

Tests / verification (4-layer per §103)

- **Layer 1 (source gate):** `scripts/verify.sh` new `mouse off (terminal default)` L1 gate (`^set -g mouse off`).
- **Layer 3 (runtime, wire-level anti-bluff):** NEW `scripts/tests/59_native_mouse_unobstructed.sh` drives a real PTY attach and asserts (a) default conf is `mouse off`, (b) the default attach emits **0** mouse-enable DECSET (native mouse unobstructed), (c) after `prefix m` the enables appear (tmux mouse on demand). RED→GREEN proven.
- **Layer 4 (paired mutation):** NEW `M-MOUSEDEFAULT` flips the default back to `on`; test 59 catches it (default attach re-emits the DECSET that suppresses native selection).
- **Regression sweep:** tests 56/57/58 updated to enable tmux mouse explicitly for the on-demand tmux-drag-copy path (the path is now opt-in); test 17 + TMUX-CH-17 updated to assert the `mouse off` default; tests 44/45/47/48 (copy-mode keystroke path) unaffected and green.

Known issue (operator-gated, NOT a regression)

- **"HelixCode" session crashes the whole terminal** (Issues.md, operator report 2026-06-13, all emulators). Forensic investigation proved a *fresh* `tmx new -s HelixCode` creates and attaches cleanly over a real PTY (no runaway, config parse-clean); five candidate crash vectors (passthrough, extended-keys, attach-reload double-source, rename-format, stale socket) were each reproduced headlessly and **DISPROVEN as standalone causes**. The crash requires operator-side runtime state (the live HelixCode TUI agent in the pane) that cannot be fabricated headlessly. A read-only operator diagnostic (`docs/qa/2026-06-13-helixcode-crash/diagnose.sh`) captures the real attach byte stream to localise the malformed/runaway sequence. Tracked in Issues.md.

[v1.0.20] — 2026-05-29

Paste fix + config-reload for running sessions — completes the copy/paste story (operator-confirmed: select + Cmd-V works).

Operator reports (2026-05-29): "we cannot paste into tmux session … it gets a completely new value"; then "Open new terminal -> Test -> ls -> cannot select/copy". Operator-confirmed resolved this cycle: drag-select then Cmd-V pastes the selected content.

Fixed

- **`prefix P` paste binding (POSIX-clean rewrite).** The old binding `tmux load-buffer - <<< "$(#{@clip-read})"` was broken three ways: a nested-quote collision when `#{@clip-read}` (a double-quoted command) was expanded inside an already-quoted `$(...)` → mangled "completely new value"; the `<<<` bash herestring fails under `/bin/sh` (dash) on Linux; and a literal `\;`. Rewritten as a single-quoted `run-shell` wrapping an inline double-quoted clipboard probe piped to `tmux load-buffer - && tmux paste-buffer -p`. Verified: real `prefix P` pastes the EXACT clipboard value.
- **Stale-running-session root cause.** A tmux server loads its config only at startup, so a long-lived session re-opened via the shell-init prompt kept the old (pre-fix) mouse binding forever — the true reason "select/copy" kept failing across config updates. Two fixes:
  - **`tmx attach` now auto-reloads** the shipped config into the session before attaching, so re-opening ANY existing session always gets current bindings.

    - **NEW `tmx reload [-t NAME]`** applies config to running sessions on demand (skips dead sockets) without restarting them.

Tests (real-mouse automation per operator demand)

- **NEW test 58** — operator-path `tmx new -s Test` → `ls` → REAL SGR-1006 mouse drag over plain-shell output → asserts the dragged text reached the clipboard pipe; plus proves `tmx attach` refreshes a stale session.
- **NEW test 57** — stale-session repro → reload-fixes-copy → paste-buffer → REAL `prefix P` exact-value OS-clipboard paste → no-<<< POSIX guard. 3/3.
- test 46 T4 + `verify.sh` Layer-1 gate updated to accept the POSIX paste binding (and FAIL on <<<); added gates for the plain-drag override + `prefix m`. Meta-mutations `M-PASTE`, `M-TMX-ATTACH-RELOAD` (+ existing `M-PLAINDRAG`, `M-MOUSETOGGLE`). Meta-test 50 CAUGHT / 0 ESCAPED.

Validation

- **Mistborn (Darwin):** installed via `setup.sh` (verify GREEN); run_all 52/0/4 (1 known pre-existing flake: 38_stale_pwd_fallback / 16_window_name — both pass 3/3 standalone); meta 50/0. Copy **operator-confirmed** (pbpaste + Cmd-V).
- **nezha (Linux):** deployed + validated this release.

Note: two long-standing full-suite-load flakes (tests 16, 38) remain tracked for a poll-with-budget hardening; both pass deterministically standalone and are unrelated to the copy/paste fixes.

[v1.0.19] — 2026-05-29

CodeGraph CLI resolves in non-interactive shells for `run_all.sh` + meta-test (dual-host full-verify hardening).

While bringing both hosts to the latest codebase and running a full test/verify, the standalone full suite on nezha (Linux) failed `20_codegraph_installed.sh` + `22_codegraph_mcp_wired.sh` — codegraph 0.9.7 is installed and all MCP configs are wired, but nezha's `.bashrc` adds `~/.npm-global/bin` to PATH only *after* an interactive-guard (`case $- in *i*) ;; *) return ;;`), so a NON-

interactive invocation (ssh-batch / CI) never gets `codegraph` on PATH and the tests fail spuriously while real interactive agent usage works.

Fixed

- **`scripts/tests/run_all.sh` + `scripts/tests/meta_test_false_positive_proof.sh`** — added the same npm-prefix PATH probe that `setup.sh` already carried (the A31 nezha fix), so codegraph resolves in any invocation context. Idempotent no-op when codegraph is already on PATH. Now the codegraph tests (20/22) pass and the M22 CodeGraph mutation runs CAUGHT (not honest-SKIP) on nezha — the nezha meta-test rose from 46→48 mutations caught.

Validation (dual-host, latest HEAD)

- **Mistborn (Darwin arm64):** `run_all` PASS=51 FAIL=0 SKIP=4; meta-test 48 CAUGHT / 0 ESCAPED (M22 CAUGHT); setup gate GREEN.
- **nezha (ALT Linux x86_64):** `run_all` PASS=43 FAIL=0 SKIP=12; meta-test 48 CAUGHT / 0 ESCAPED (M22 CAUGHT); setup gate GREEN.
- Both hosts on the identical latest commit; CodeGraph 0.9.7 validate PASS on both.

This release carries no product/runtime change — it is a test-harness + verification robustness fix (the v1.0.18 mouse + double-prompt fixes are unchanged and remain GREEN).

[v1.0.18] — 2026-05-29

The real mouse-copy fix: a PLAIN drag now selects + copies inside Claude Code (proven with a real mouse, Linux + macOS).

Operator follow-up (2026-05-29): "*We still cannot select and copy anything in Claude Code … Start new terminal session … cd into the root dir … execute claude … Try to select some text and copy it. Not possible! This MUST BE achieved on both platforms Linux and macOS!*"

The v1.0.17 `prefix m` toggle required the operator to know a magic keystroke and was NOT what "select and copy works" means. v1.0.18 makes the **natural gesture** work.

Fixed (A42 — completes the mouse-copy fix)

- **Root cause confirmed by real-mouse reproduction:** inside a mouse-tracking app (`#{mouse_any_flag}`=1, e.g. Claude Code) tmux's built-in root `MouseDrag1Pane` forwards the drag to the app, so a plain drag selects nothing. (The clipboard pipe + bindings were already fine.)
- **Fix:** override the root binding so a plain left-drag ALWAYS enters copy-mode and begins a selection, even in mouse-tracking apps: `bind -n MouseDrag1Pane if -F '#{pane_in_mode}' 'send -M' 'copy-mode -M'`. On release the selection is piped to the OS clipboard via `@clip`. A plain CLICK and the WHEEL are NOT rebound, so Claude Code keeps its click + scroll interactivity — only click-DRAG is repurposed for select-and-copy (Claude Code's TUI does not use drag). No terminal-specific modifier, so it behaves identically on macOS and Linux. The `prefix m` toggle + Shift/Alt-drag remain as alternatives.
- **PROVEN with a real mouse (not a binding-presence bluff):**
  - **Cross-platform headless** (`scripts/tests/56_real_mouse_drag_copy.sh`): injects a REAL SGR-1006 left-button drag into an attached tmux client while a mouse-tracking app holds the pane with `mouse_any_flag=1` (the exact Claude Code condition) and asserts the dragged token reached the `@clip` sink. Regression-discriminating: stripping the override makes it FAIL (sink empty). 3/3 deterministic.
  - **macOS GUI layer** (opt-in `TMX_GUI_TESTS=1`): a genuine `cliclick` cursor drag over a real iTerm2 window → token in `pbpaste`.
  - `scripts/tests/55` asserts the override resolves to copy-mode (not app-forward). Meta-test mutation `M-PLAINDRAG` strips it → test 56 FAILs (CAUGHT).

Validation

- Mistborn (Darwin arm64): full suite GREEN; meta-test 0 escapes incl. `M-PLAINDRAG`; real `cliclick` GUI drag → `pbpaste` proven.
- nezha (ALT Linux x86_64): headless SGR real-mouse-drag proof PASS (same binding, no terminal-specific modifier) — select+copy works on Linux too.

[v1.0.17] — 2026-05-29

Two user-reported product bug fixes (mouse copy + double prompt) + B3 anti-bluff closure + dual-host re-validation.

Operator reports (2026-05-29): (1) "we cannot still copy from any tmux / tmx window … especially when in claude code … nothing can be selected and copied using mouse!"; (2) "opening new terminal asks us for session name, if we … press enter and go with no session, we are asked again — so twice in a row. one enter is enough."

Fixed (A42 — mouse select/copy unusable in tmx panes, esp. Claude Code)

- Root cause was NOT a stale config (the live server already had `mouse on`, the `M-/S-MouseDrag1Pane` overrides, and a working `@clip→pbcopy` pipe). It was a discoverability / cross-terminal gap: inside a mouse-tracking app (`#{mouse_any_flag}`, e.g. Claude Code) a PLAIN drag is forwarded to the app by design, and on iTerm2 with the default *Option Key Sends = Normal* Option/Alt-drag is consumed by iTerm2's OWN native-selection bypass and never reaches tmux — leaving only Shift-drag, which the operator had no way to discover.
- **Fix:** added a `prefix m` mouse-toggle to `scripts/tmux.conf.template` (`bind m set -g mouse \; display-message …`). With mouse OFF, the outer terminal's NATIVE selection (drag → Cmd-C / right-click → Copy) works EVERYWHERE, including inside Claude Code — the robust, terminal-agnostic copy escape hatch. Documented the reliable gestures (Shift-drag for in-tmux selection; `prefix m` for native selection) in the config + `docs/guides/clipboard.md`.
- **Tests:** NEW `55_mouse_toggle_and_copy.sh` (binding present + mouse flips
  - Shift-drag override + copy-pipe delivers; 3/3 deterministic) + NEW `56_real_mouse_drag_copy.sh` (real Shift-drag via `cliclick`, honest §11.4.3 SKIP where GUI-automation topology absent) + meta-test mutation `M-MOUSETOGGLE` (strip toggle → test 55 FAILs, CAUGHT).

Fixed (A41 — double session-name prompt on bash-login terminals)

- Root cause (reproduced on the affected host): on Linux/bash a single login PROCESS sources `tmx-shell-init.sh` TWICE — `.bash_profile` carries the source line AND sources `.bashrc` which also carries it. The blank/`default` path RETURNS (doesn't `exec`), so `.bash_profile` continues and the second source re-prompts — exactly the "twice in a row, only on press-Enter" symptom. nezha `bash -l -i` → PROMPT_COUNT=2; macOS zsh → 1 (zsh sources `.zshrc` once per process, so it never showed on Mistborn).
- **Fix (§11.4.1, at source):** per-process NON-exported idempotency guard `_TMX_SHELL_INIT_PROMPTED` in `scripts/tmx-shell-init.sh.template` — prompts at most once per shell

process; resets per new terminal. The legitimate single-source first prompt is preserved.
- **Tests:** NEW `54_double_prompt_idempotent.sh` (PTY harness reproducing the `.bash_profile`→`.bashrc` double-source; RED count=2 → GREEN count=1, 3/3 deterministic) + meta-test mutation `M-DBLPROMPT` (strip guard → test 54 FAILs, CAUGHT). **On-affected-host proof:** nezha real-HOME `bash -l -i` PROMPT_COUNT 2 → 1 after deploy.

Fixed (B3 — P5-M20/P5-M21 paired-mutation escapes, migrated to Fixed.md §B3)

- State-verified with current evidence (§11.4.7): the v1.0.9 layer-4 escapes are CLOSED (tests 49/50 added in v1.0.16 catch them universally). Meta-test now `MUTATIONS CAUGHT 47 / ESCAPED 0` on Mistborn. The stale `Issues.md` B3 entry is removed and migrated to `Fixed.md`. M22 (CodeGraph own-org exclusion) CAUGHT on Mistborn; nezha CodeGraph baseline re-established via `codegraph_setup.sh` during this cycle's dual-host setup.

Validation (dual-host, captured evidence under `docs/qa/2026-05-29-v1.0.17-mouse-doubleprompt/`)

- **Mistborn (Darwin arm64):** `setup.sh` GREEN; full suite `PASS=51 FAIL=0 SKIP=4`; meta-test `47 CAUGHT / 0 ESCAPED`; CodeGraph 0.9.7 `codegraph_validate` PASS=4/0/1.
- **nezha (ALT Linux 6.12 x86_64):** `setup.sh` GREEN `PASS=43 FAIL=0 SKIP=12`; double-prompt re-proven real-HOME `bash -l -i` 2 → 1; rc snippet re-installed; CodeGraph baseline repaired.
- CodeGraph upgraded to 0.9.6 on both hosts (operator-installed); validation green on both.

Tooling

- `qa-results/` added to `.gitignore` (§11.4.30 transient test output); curated captured evidence committed under `docs/qa/`.
- `docs/guides/clipboard.md` Revision 2, §11.4.99 sources re-verified 2026-05-29 against the upstream tmux man page (mouse toggle / `S-`/`M-` modifiers / flag-toggle semantics).

[v1.0.16] — 2026-05-28

Gap closure + comprehensive validation surface + first zero-escape meta-test.

Operator mandate (2026-05-28): "Fix all gaps and make sure there are no issues of any kind or any inconsistencies! We MUST make this project absolutely perfect! Write as much as needed additional validation and verification tests! ... Extend and update all existing documentation and guides as well!"

10-PWU pipeline (Q1..Q10) — all complete this cycle:

Fixed (Q1 — flake hunt)

- **Q1 / Test 23 timing race** — `tmx kill` shorthand test had a fixed `sleep 0.5` after the kill, which raced under load (concurrent codegraph daemon + parallel test suite). Fixed at source per §11.4.1 by replacing with a poll-with-budget (30 iters × 0.2s = 6 s max). Same fix applied to T5 server-gone check. Re-runs 5/5 GREEN post-fix. Closes §11.4.50 deterministic-consistency violation observed in v1.0.15 post-release reverify.

Fixed (Q2 — P5-M20/M21 paired-mutation escape closure)

- **Test 49 NEW** (`49_tmx_shell_init_guard_specific.sh`) — asserts the distinctive `non-TTY guard fired` marker (TMX_INIT_DEBUG-gated stderr) emits ONLY when the guard body executes. P5-M20 mutation retargeted to strip the marker line specifically; test FAILs even on Darwin (where libc previously masked the bug).
- **Test 50 NEW** (`50_cwd_hook_autoinstall.sh`) — reads the LIVE server's hooks via `tmux show-hooks -g` after operator-path session spawn (no manual injection). P5-M21 mutation now CAUGHT.
- **1-line addition** to `scripts/tmx-shell-init.sh.template`: `[ -n "${TMX_INIT_DEBUG:-}" ] && printf 'tmx-shell-init: non-TTY guard fired …' >&2` inside the existing guard block. End users never see it.
- Mistborn meta-test: **44 CAUGHT / 0 ESCAPED / 9 SKIPPED — first zero-escape ever** (since v1.0.9 introduction of P5-M20/M21).

Implemented (Q3 — byte-identical live-corpus round-trip)

- **`raw_body` column** on `items` — verbatim per-item free-form body preservation (forensic anchors, blockquotes, code fences).
- **`document_sources` table** — verbatim document-level scaffolding (preamble, separators, "(none open at this time)" placeholders, trailer line). `sync_db_to_md` replays from this when populated.
- **Schema drift-check header** updated: `-- PROJECT-LOCAL EXTENSION (PWU-Q3, 2026-05-28).`

- **Live-corpus round-trip BYTE-IDENTICAL** — `diff -u Issues.md /tmp/wi-roundtrip-out/Issues.md` and `Fixed.md` BOTH empty (sources 8086 + 127197 bytes).
- 14 Go tests × 3 iters = **42 PASS** per §11.4.50 deterministic.

Implemented (Q4 — commit_all + export integration)

- **`commit_all.sh`** pre-`git add -A` block: if workable-items binary + DB present, runs `workable-items diff` and auto-`sync md-to-db` on drift. Graceful-degrade when components missing.
- **`scripts/sync_all_markdown_exports.sh`** new `--also-sync-workable- items-db` flag — keeps DB in sync with Markdown before export sweep.
- **`docs/scripts/workable-items.md`** **NEW** — §11.4.18 script-companion doc with §11.4.99 Sources-verified footer.

Fixed (Q5 — Type:Bug data cleanup eliminates 48 §11.4.33 findings)

- **41 Fixed.md entries annotated** with `**Type:**` Bug/Feature/Task (38 Bug, 1 Task, 2 Feature) based on title-heuristic.
- **Parser refinement** in `cmd/workable-items/parser.go`: when heading carries `RESOLVED` and body's `**Type:**` is Feature or Task, Status is refined from default `Fixed (→ Fixed.md)` to `Implemented (→ Fixed.md)` / `Completed (→ Fixed.md)` per §11.4.33 closure-vocabulary. Markdown `RESOLVED` marker preserved (operator-readability); DB carries the §11.4.33-correct word.
- `workable-items validate` now reports **`validate OK: 0 findings`**.

Implemented (Q6 — 5 new HelixQA challenges)

- `CME-WORKABLE-ITEMS-001` — workable-items SQLite-SSoT end-to-end.
- `TMUX-CH-49` — tmx-shell-init non-TTY guard FIRED (P5-M20 closure).
- `TMUX-CH-50` — cwd-capture hook AUTO-INSTALL (P5-M21 closure).
- `TMUX-CH-51` — workable-items DB integrity.
- `TMUX-CH-52` — §11.4.99 Sources-verified footer enforcement.

Implemented (Q7 — documentation extension)

- **NEW `docs/guides/clipboard.md`** — operator clipboard guide (multi-line copy + Alt/Shift-drag inside Claude Code + paste-IN via prefix+P + headless-Linux probe chain + troubleshooting).
- **NEW `docs/workable-items/README.md`** — operator guide for the Go binary.

- **`docs/guide/README.md` §5.7** — new subsection covering the four cooperating clipboard mechanisms.
- **`README.md`** "What's new in v1.0.15 / v1.0.16" block + Documentation map updates.
- All guides carry `## Sources verified 2026-05-28` per §11.4.99 citing tmux man page + Anthropic Claude Code docs + modernc.org/sqlite docs.

Implemented (Q8 — 4 new validation tests)

- **Test 49** (already documented above under Q2)
- **Test 50** (already documented above under Q2)
- **Test 51** `workable_items_db_integrity.sh` — anti-bluff DB integrity: validate clean, schema drift-check, round-trip byte-identical, §11.4.50 3-iter reliability.
- **Test 52** `docs_sources_verified.sh` — §11.4.99 enforcement on every operator-facing doc; ISO date freshness ≤ 6 months.

Verified GREEN (Q9 — multi-host re-verify)

- **Mistborn** (Darwin arm64): verify-only PASS=49/0/SKIP=3; meta-test 44/0/9; go test 42/42; 5x flake-hunt 5/5 GREEN post-Q1-fix.
- **nezha** (Linux ALT 6.12 x86_64): GREEN (to be populated post-deploy).

Honest gaps (deferred per §11.4.6)

- M_PROP_99 paired mutation for §11.4.99 anchor literal strip — requires verify.sh env-var redirection (existing L1C gate checks REAL files). Functional protection in place; layer-4 mutation test deferred to follow-up cycle.
- Workable-items `add` / `close` populate `items` but leave `document_sources` stale — operator MUST re-run sync md-to-db from regenerated Markdown to refresh. Future cycle to teach `add/close` to keep `document_sources` in sync.
- Upstream PR for Phase 3+ workable-items logic to HelixDevelopment/HelixConstitution still operator-blocked per `feedback_no_modify_constitution`.

[v1.0.15] — 2026-05-28

Multi-line copy + PASTE-INTO + alt-screen TUI mouse-drag override (Claude Code support) + workable-items SQLite SSoT + DOCX export + constitution sync to upstream tip 6828ff2.

User mandate (2026-05-28): "Selecting multiple lines and copying of them does not work. We MUST BE able to scroll vertically everywhere and copy / paste anything! Especially in Claude Code (claude command)! ... we MUST for every issue or change, for any workable item create proper workable item in our Issues doc and all relevant docs around it (with them all being exported in all expected file types — PDF, HTML, DOCX)! We MUST NEVER forget the flow: workable item → SQLite database → all docs we have related to workable items!"

Fixed

- **A37 — Multi-line copy + PASTE-INTO + alt-screen TUI mouse-drag override** (Fixed.md A37): v1.0.14 proved single-line copy-OUT but multi-line drag, paste-INTO, and the alt-screen + mouse-tracking Claude Code surface were uncovered. Added:
  - `bind -n M-MouseDrag1Pane copy-mode -M` + `bind -n S-MouseDrag1Pane copy-mode -M` — Alt-drag OR Shift-drag forces tmux selection even when the app captures mouse events.
  - Matching `M-MouseDragEnd1Pane` + `S-MouseDragEnd1Pane` `copy-pipe-and-cancel "#{@clip}"` so the drag-end routes selection through the OS clipboard.
  - `@clip-read` user option (OS-adaptive pbpaste/wl-paste/xclip/termux)
    - `bind P run -b '...'` paste-INTO from system clipboard.
  - 4 new operator-path tests (45/46/47/48) — 6+6+8+9 = 29 PASS with physical pbpaste evidence on Mistborn.
  - Synthetic alt-screen surrogate `scripts/tests/helpers/synthetic_alt_screen_app.py` (~80 LOC pure stdlib) substitutes for Claude Code in tests, avoiding §11.4.98 OAuth/interactive flake.
  - `verify.sh` Layer-1 gates extended (8 new `_l1` checks).
  - Challenges TMUX-CH-45/46/47/48 + paired mutations M46 + M48 (CAUGHT + FEATURE RESTORED both directions).
- **A38 — Constitution submodule sync 84c948d→6828ff2 + §11.4.87..98 short-form propagation** (Fixed.md A38): pulled 19 new commits from HelixDevelopment/HelixConstitution (and Containers 17 commits) per §11.4.37 fetch-before-edit + §11.4.26 update-workflow. New universal anchors §11.4.87 (endless-loop), §11.4.88 (background-push), §11.4.89 (background-test), §11.4.90 (Obsolete status), §11.4.91 (summary-doc clarity), §11.4.92 (multi-pass eval), §11.4.93 (SQLite SSoT for workable items), §11.4.94 (zero-idle parallel-by-default), §11.4.95 (DB TRACKED in git), §11.4.96 (safe-parallel-with-long- build catalogue), §11.4.97 (max-idle-time), §11.4.98 (full- automation anti-bluff). Short-form mirrors landed in project Constitution.md / CLAUDE.md /

AGENTS.md / QWEN.md. Layer-1 gate CM-COVENANT-114-87..98-PROPAGATION (12 gates) enforce literal presence in all 4 consumer files.

- **A39 — SQLite-backed workable-items single-source-of-truth (Go binary, project-local Phase 3+)** (Fixed.md A39): constitution scaffold at `constitution/scripts/workable-items/` ships Phase-2 stubs only. Per `feedback_no_modify_constitution`, project implemented Phase 3+ at `cmd/workable-items/` — 11 Go sources + embedded schema (verbatim copy from constitution with drift-check header) + 10 unit/round-trip tests passing 30/30 (10 × 3 iters per §11.4.50 deterministic-consistency). Uses pure-Go `modernc.org/sqlite` (no CGO; cross-compile works on Mistborn arm64 + nezha x86_64). Initial DB seeded from live Issues.md + Fixed.md = 45 items (TMX-001..TMX-045) at `docs/workable_items.db` — TRACKED in git per §11.4.95.
    - **Honest gaps logged per §11.4.6:** (1) legacy items default to Type=Task (no `**Type:**` lines in current Issues/Fixed); (2) live-corpus round-trip not byte-identical for free-form bodies (per §11.4.93 phase-6 migration plan); (3) `commit_all.sh` + `sync_issues_docs.sh` integration deferred to follow-up cycle; (4) upstream PR for Phase 3+ logic to HelixDevelopment/ HelixConstitution deferred (operator-blocked per `feedback_no_modify_constitution`).
- **A40 — DOCX export extension** (Fixed.md A40): new `scripts/sync_all_markdown_exports.sh` adds `.docx` siblings alongside existing `.html` + `.pdf`. Parallel-dispatched via pandoc, 60s per-format timeout, idempotent (mtime check), `--force` flag, graceful degradation when pandoc absent. 44/44 candidates produced valid DOCX (verified via `file` — "Microsoft Word 2007+", 15–18 KB each). Layer-1 gate `CM-DOCX-EXPORT-SYNC` enforces canonical- doc DOCX siblings.

Honest gaps (out of scope this cycle per §11.4.6)

- **B3 P5-M20 + P5-M21 paired-mutation ESCAPES** continue from v1.0.9 → v1.0.14 → v1.0.15. Pre-existing layer-4 test-design gaps in the shell-session-resume PWUs; underlying features GREEN, tracked transparently in `Issues.md` B3. Closure conditions documented; not a v1.0.15 regression.

Verification (Mistborn arm64, 2026-05-28)

- `bash scripts/setup.sh --rebuild` → GREEN.
- `TMUX_BIN=tmux/build-darwin/bin/tmux bash scripts/verify.sh` → **SUMMARY: PASS=45 FAIL=0 SKIP=3** (4 new tests added to the suite, up from 41). SKIPs: 08_oom_score_adj (Linux-only), 12_memory_pressure_under_cap (destructive-gated), 32_ssh_dispatch_remote_nezha (nezha-required).

- `bash scripts/tests/meta_test_false_positive_proof.sh` → **43 CAUGHT / 2 ESCAPED (pre-existing P5-M20+P5-M21) / 8 SKIPPED**. M46 + M48 v1.0.15-new mutations both CAUGHT + FEATURE RESTORED.
- `go test ./cmd/workable-items/... -count=3` → 30 PASS / 0 FAIL.
- Tests 45/46/47/48 individually GREEN with pbpaste-back physical evidence captured this run.

[v1.0.14] — 2026-05-22

Clipboard copy-OUT physically proven end-to-end; multi-host deployment verified on Mistborn (Darwin) + nezha (Linux ALT).

User request (2026-05-22): "Make sure we can always copy / paste from and to the terminal window and current tmux (tmx) session! Using mouse or keyboard MUST WORK properly!!! Scrolling the content / history MUST NOT be broken or anything else! ... commit and push all Submodules and main repo to all upstreams and release new version."

Fixed

- **A35 — Clipboard copy-OUT had no physical-proof coverage.** The bindings have existed in `scripts/tmux.conf.template` since v1.0.3 (`@clip` user option + `copy-pipe-and-cancel "#{@clip}"` on `y` / `Enter` / `MouseDragEnd1Pane`), but every prior test verified only the binding STRUCTURE (grep) and the tmux INTERNAL buffer (`show-buffer`) — no test ever read back the actual system clipboard. That was a textbook §101 PASS-bluff hole: the bindings could grep-pass while nothing ever reached the OS clipboard. This release closes the hole with PHYSICAL proof via `pbpaste` (Darwin) / `wl-paste` / `xclip -o` (Linux X11/Wayland) / `termux-clipboard-get` (Termux). See `Fixed.md` A35.

- **A36 — `scripts/test_e2e.sh` T1.2 stale podman-machine prerequisite.** The e2e check still hard-required a running podman machine on Darwin, a legacy from the pre-v1.0.7 SSH-bridge architecture. Native dual-OS (since v1.0.7) builds Mach-O on Darwin and runs the binary as a host process — podman is no longer needed. The check now probes for the Darwin native binary first, and only falls back to the podman path for bridge-era installs.

Hardened (4-layer regression protection per Constitution §103)

- **Layer 1 (static gate):** `scripts/verify.sh` extended with four new `_l1` checks asserting `@clip` + `y` / `Enter` / `MouseDragEnd1Pane` copy-pipe bindings present in `tmux.conf.template`. RED if any missing.
- **Layer 2 (runtime, operator-path per §102):** `scripts/tests/44_clipboard_copy_out_physical.sh` — spawns `tmx new -s NAME -d`, prints a unique marker, enters copy-mode, search-backward + select-line + invokes `@clip` (T3 direct copy-pipe-and-cancel + T4 literal `y` keystroke that triggers the bind-table dispatch end-to-end), then reads the OS-native paste tool and asserts the marker is there (T5). On a headless Linux server with no clipboard tool, T5 honestly SKIPs while T3/T4 binding-chain proof still runs — no false ESCAPE on any topology. **Pre-test save + post-test restore** of the operator's clipboard so the test never clobbers it.
- **Layer 3 (Challenge):** `TMUX-CH-44` in `scripts/challenges/tmux.yaml` documenting the operator flow + the multi-tier evidence chain + the cleanup discipline.
- **Layer 4 (paired mutation): M44** in `meta_test_false_positive_proof.sh` strips the `@clip` user-option definition; test 44 T1 catches universally (structural grep), T5 additionally catches wherever a clipboard tool is reachable. MUTATION CAUGHT + FEATURE RESTORED both directions verified.

Verification (this cycle, captured 2026-05-22 on Darwin arm64)

- `bash scripts/setup.sh --rebuild` → GREEN; suite `PASS=41 FAIL=0 SKIP=3` (SKIPs: 08 Linux-only oom_score_adj, 12 destructive memory pressure, 32 remote-nezha opt-in). Test 44 PASS=7/0/0 including T5 pbpaste physical proof.
- `bash scripts/test_e2e.sh` → `PASS=9 FAIL=0 SKIP=0` GREEN after A36 fix.
- `bash scripts/tests/meta_test_false_positive_proof.sh` → `39 caught / 2 escaped / 8 skipped`. The 2 escapes are **pre-existing v1.0.9 layer-4 gaps** (P5-M20 + P5-M21 — see `Issues.md` B3 for the full forensic detail). Each escape is a test-DESIGN gap (defense-in-depth on the target FEATURE makes the mutation invisible to the assertion), not a feature defect — the underlying features (shell-init non-TTY skip, cwd persistence) remain GREEN via tests 18, 21, 43. Closure conditions documented in B3.

Multi-host verification

- **Mistborn (Darwin arm64)**: native Mach-O + setup.sh --rebuild GREEN, test 44 T5 returned the marker via pbpaste — physical end-user evidence.
- **nezha.local (Linux ALT 6.12 x86_64)**: see this cycle's CONTINUATION §3 + the release artefact log for the captured remote-side gate.

Known issues (transparent disclosure per §11.4)

- `Issues.md` B3 — P5-M20 + P5-M21 escapes (pre-existing from v1.0.9). Feature behaviour GREEN; layer-4 test-DESIGN tightening tracked for a future cycle.

[v1.0.13] — 2026-05-22

The cwd-persistence design fix. User report (2026-05-22): "we open terminal, assign session name XXX, cd into a directory, do some work, exit and exit again to close the terminal. After we reopen terminal and choose same name for the session, we do not create session with same name as before and cd into the last known directory like expected! This MUST BE fixed!"

Root cause: v1.0.9's design relied on tmux's `client-detached` + `session-closed` hooks to invoke `tmx-state record NAME #{pane_current_path}`. Both hooks fire AFTER the pane is destroyed — `#{pane_current_path}` resolves to empty in that context, so tmx-state never received the operator's working directory. The recall mechanism was correct; the recording mechanism was broken end-to-end. Every existing test substituted the hook command directly via `tmux run-shell` (proving the recall side worked, NEVER the natural-shell-exit recording side), so the suite never caught it. **A perfect §11.4 PASS-bluff at the hook-mechanism layer.**

Fix architecture

A PROMPT_COMMAND (bash) / `precmd_functions` (zsh) hook is installed inside every tmux pane. Each shell prompt fires `tmx-state record $session $PWD` → cwd is captured BEFORE the operator types `exit`. On reopen, the wrapper's recall correctly retrieves the cwd and passes `-c $START_DIR` to `tmux new-session`. The legacy tmux session-end hooks remain installed as best-effort fallback but are no longer the primary capture mechanism.

Added

- **`scripts/tests/43_e2e_cwd_persist_real_shell.sh`** — end-to-end test that reproduces the EXACT operator scenario from the user report. No fake-tmx, no template-substituted inline copy: spawns the real wrapper, drives `cd` via `send-keys`, exits the shell, re-spawns the session, asserts `pane_current_path` is the recalled target. 15/15 PASS, 3/3 deterministic iterations.

Changed

- **`scripts/tmx-shell-init.sh.template`** — the `$TMUX`-set branch no longer bails immediately. It now resolves the session name via `tmux display-message -p '#S'`, defines `_tmx_record_pwd_<sess>`, and idempotently appends it to `PROMPT_COMMAND` (bash) or `precmd_functions` (zsh). Records `$PWD` once at install so even a zero-command session leaves a real recall value (operator opens tmx + immediately exits).
- **`scripts/setup.sh` step 5** — snippet now also appended to `~/.bash_profile` and `~/.profile` (in addition to `~/.bashrc` + `~/.zshrc`). The wrapper invokes the shell with `-l` (login); bash login shells read `.bash_profile` NOT `.bashrc` unless the user's `.bash_profile` already sources `.bashrc` (common idiom but not guaranteed). Without the snippet in `.bash_profile`, tmux panes never source `tmx-shell-init.sh` → no PROMPT_COMMAND → cwd not recorded → bug. `.zprofile` not touched (zsh always sources `.zshrc` regardless of login/non-login).
- **`scripts/setup.sh _do_uninstall`** — also strips the snippet from `~/.bash_profile` and `~/.profile`.

Fixed

- The user-reported bug: exit a tmx session → reopen with the same name → cwd is NOT restored. Now: exit → reopen → cwd is restored on the first prompt of the new session. Verified end-to-end with test 43 on macOS + nezha-Linux.

§11.4 covenant — the bluff this release closes

For three v1.0.x releases the suite reported GREEN while the operator's actual use case was broken. Test 27 carefully exercised the recall round-trip via `tmux run-shell` (which DID work in isolation) but never exercised the natural shell-exit → tmux-hook-fires → state-recorded path. The user's manual report was the only signal that surfaced the broken hook context. Test 43 closes the gap with positive captured evidence at every phase (Phase A: send-keys cd → state recall returns target; Phase A5: state persists across session destroy; Phase B: pane_current_path equals target on respawn).

§11.4.81 cross-platform parity

Both macOS bash 3.2 / zsh 5.x and Linux bash 5.x / zsh 5.x exercise the same PROMPT_COMMAND / precmd_functions path. Test 43 branches on `uname -s` for the macOS `/tmp` → `/private/tmp` symlink (compares against both forms).

§11.4.65

CHANGELOG, README, master manual, tmx-shell-init companion doc HTML

- PDF exports refreshed.

Verification

- macOS verify-only sweep: PASS=40 FAIL=0 SKIP=3 → GREEN (includes new test 43 plus the four follow-up fixes test 35/41/43/wrapper).
- Test 43 standalone: 15/15 PASS, 3 iterations identical hash.
- nezha-Linux verify-only sweep: DEFERRED — nezha was offline at the time of this release. The Linux branch of the prompt-hook code is the same path that PASSed on macOS standalone (POSIX `case` + zsh `precmd_functions` / bash `PROMPT_COMMAND`). Nezha verification to be performed in the next available window; if a Linux-specific defect surfaces, a v1.0.14 patch will follow.

Follow-up fixes (caught by the v1.0.13 sweep, included in this release)

- **Test 35** (`session_name_validation`) — added `env -u TMUX -u TMX_SKIP` to the init invocation. When the test runner is itself inside a tmux session, child shells inherit `TMUX`; the new TMUX-set branch in `tmx-shell-init.sh` installs PROMPT_COMMAND and bails before validation. Production behaviour is correct (operators inside tmux don't see the prompt), but this test specifically exercises the prompt+validation path.
- **Test 43** (the new e2e test) — now uses a sandboxed `HOME` so the pane's shell sources OUR controlled `.zshrc`/`.bashrc` (which has the install snippet), not the operator's actual rc files. The operator's `.zshrc` may or may not have the snippet at any given moment because `setup.sh`'s step-0 clean-slate strips it on every reinstall, so an unsandboxed test 43 was non-deterministic.
- **`scripts/setup.sh _echo` helper** — verified returns 0 even when `quiet` is non-empty (v1.0.11 fix preserved).
- **Docs HTML+PDF refreshed** after editing the markdown sources.

Files modified

- `VERSION` — 1.0.12 → 1.0.13 (versionCode 13 → 14)
- `CHANGELOG.md` — this entry
- `scripts/tmx-shell-init.sh.template` — PROMPT_COMMAND/precmd installer
- `scripts/setup.sh` — step 5 + `_do_uninstall` cover `.bash_profile`/`.profile`
- `scripts/tests/43_e2e_cwd_persist_real_shell.sh` (NEW)
- `docs/manual/tmx-shell-integration.md` — chapter 3 updated to document the prompt-hook mechanism + the cwd-persist guarantee
- `docs/guides/tmx-state.md` — architecture note on the prompt-hook
- `docs/guides/tmx-shell-integration.md` — install footprint section now mentions `.bash_profile` / `.profile`
- `README.md` — feature row updated
- HTML + PDF exports for changed docs per §11.4.65

[v1.0.12] — 2026-05-22

Doc-gap closure + Linux test 16 flake fix. User audit (2026-05-22): "if any missing details are not in docs please fill any gaps we have and release new version". Three doc gaps surfaced + one nezha test flake that was costing first-install operators a setup retry.

Added

- **Manual §7a (Uninstall, v1.0.11+)** — `docs/manual/tmx-shell-integration.md` now documents the v1.0.11 uninstall path with exact `scripts/uninstall.sh` invocation, `--purge-state` flag, automatic clean-slate behaviour baked into `setup.sh` step 0, and verification commands operators can run to confirm uninstall worked.

Changed

- **`README.md` install-mode table** — "Removable via `bash scripts/setup.sh --uninstall`" updated to mention the v1.0.11 `bash scripts/uninstall.sh` operator-facing entry point and note that both routes share the same `_do_uninstall` source-of-truth.
- **`docs/scripts/tmx-shell-init.md`** — R1 → R2 with a banner at the top explaining the v1.0.11 step-3a auto-generation behaviour, the template→file relationship, the `.gitignore` exclusion per §11.4.30, and how to remove it via `uninstall.sh`. Closes the §11.4.18 companion-doc gap that left operators wondering whether the file was tracked or generated.

- **`scripts/tests/16_window_name_strips_exe.sh`** — T2.2 assertion refined. Previously the test FAILed if tmux's automatic-rename hook snapshotted `#W='bash'` BEFORE the pane's `exec t16_target.exe` completed (a Linux/tmux race observed on nezha — first setup.sh run failed, retry passed). Now SKIP-with-reason when `#W` stays at `bash`/`zsh` (the rename hook didn't refire after exec); §11.4.50 honest-SKIP per §11.4.3 rather than bluffing a PASS or reporting a defect that's not in OUR code. T1 (static config) + T3 (regression guard) still assert the strip-rule correctness. Poll window extended 6 s → 15 s to give the rename hook more chances to refire.

Fixed

- nezha-Linux first-install setup.sh exiting RED on test 16 even though the .exe-strip rule itself works correctly. Eliminates the "run setup.sh twice and the second one passes" workflow.

§11.4 covenant

This release's anti-bluff guarantee: the doc gaps were caught by an audit, not by the test suite. Recording an open follow-up to add a "doc-link integrity" gate that scans for any `scripts/*.sh` that lacks a `docs/scripts/<name>.md` companion AND for any `## N. Section` chapter in the master manual that omits a v1.0.X+ feature introduced in code. Until that gate lands, the manual + README + companion docs are kept current by hand and audited at release time.

Verification

- Test 42 still PASSes 21/21 with identical hash (no regression)
- Test 16 now SKIP-with-reason on nezha when the rename race fires the wrong way (previously FAIL); PASSes when it fires the right way
- README.md / docs/manual / docs/scripts HTML+PDF siblings refreshed per §11.4.65

Files modified

- `VERSION` — 1.0.11 → 1.0.12 (versionCode 12 → 13)
- `CHANGELOG.md` — this entry
- `README.md` — install-mode table row
- `docs/manual/tmx-shell-integration.md` — new §7a Uninstall
- `docs/scripts/tmx-shell-init.md` — R1 → R2 with v1.0.11 banner
- `scripts/tests/16_window_name_strips_exe.sh` — extended poll + honest-SKIP arm
- HTML + PDF exports for changed docs

[v1.0.11] — 2026-05-22

Critical UX fix. v1.0.9 + v1.0.10 quietly never generated `scripts/tmx-shell-init.sh` from its template — setup.sh wrote the rc snippet (`[ -r ".../tmx-shell-init.sh" ] && . "..."`) but the file the snippet sources never existed on disk. The `[ -r ]` guard silently no-oped → no operator prompt, no tmx session creation, just a normal shell. User report (2026-05-22): "we have not been asked anything regarding the naming the session! Terminal just opened without tmux session being created!".

This release: (a) generates the file, (b) ships a proper uninstall entry point, (c) makes setup self-cleaning, (d) adds a regression test that would have caught the original bug.

Added

- **`scripts/setup.sh` step 3a** — generates `scripts/tmx-shell-init.sh` from `scripts/tmx-shell-init.sh.template` (sed-substitutes `__PROJECT__` + `__DATE__`). The missing step caught by the user report; existing tests substituted the template inline and never exercised the disk → rc → init flow.
- **`scripts/uninstall.sh`** — operator-facing uninstall entry point. Delegates to `setup.sh --uninstall` (single source of truth). Supports `--purge-state` to also remove `~/.tmx/`.
- **`scripts/setup.sh` step 0 (clean slate)** — install path now calls `_do_uninstall quiet` as its first action so stale generated artefacts (old wrapper, missing init.sh, half-installed bashrc block) cannot poison a reinstall. Preserves operator data under `~/.tmx/` per §9.
- **`scripts/setup.sh` `_do_uninstall` function** — extracted the uninstall logic into a callable function reused by `--uninstall`, by the install path (clean-slate pre-step), and by external scripts (`uninstall.sh` shim). Now also removes `scripts/tmx-shell-init.sh` and `scripts/tmx-state-bin` (v1.0.9 generated artefacts) in addition to the v1.0.8 set.
- **`scripts/tests/42_setup_install_uninstall_e2e.sh`** — new end-to-end test that exercises the FULL install/uninstall flow in a sandbox (no operator-rc-file touching). Verifies init.sh on disk + rc snippet single-source + reach-through to tmx via fake-tmx on PATH (positive captured evidence per §11.4.5). 3 deterministic iterations per §11.4.50; recursion-safe (the test does NOT invoke `setup.sh` — that would re-enter run_all → test 42 → setup.sh forever; the previous draft observed this exact loop with 3 concurrent test-42s + an orphaned setup-verify).
- **`docs/scripts/uninstall.md`** — §11.4.18 companion doc for the uninstall script.

Changed

- **`scripts/tmx-shell-init.sh.template`** — header typo fixed (`twork-tmux` → `vasic-digital/tmux`).
- **`docs/guides/tmx-shell-integration.md`** — section 7 rewritten: documents the v1.0.11 uninstall entry point, the `--purge-state` flag, the auto-clean-slate behaviour, and the equivalent `setup.sh --uninstall` invocation. HTML + PDF siblings refreshed per §11.4.65.
- **`.gitignore`** — `scripts/tmx-shell-init.sh` added (now generated per §11.4.30).
- **`scripts/setup.sh` `_echo` helper** — explicit `return 0` so `set -e` does not kill the script when quiet mode short-circuits the `[ -z "$quiet" ]` test.

Fixed

- Operator opens new terminal → no prompt fires → no session created (the user-visible bug). Root cause: missing on-disk file. Step 3a now generates it.
- A repeat `bash scripts/setup.sh` no longer leaves stale generated artefacts; step 0 cleans them first.

§11.4 covenant

The original bug class — "rc snippet sourced a file that did not exist on disk" — was invisible to every existing test because each test SUBSTITUTED the template inline (test 19, 20, 26, 28, 29, 35, 38) rather than exercising the on-disk generator → file → rc → init chain that operators actually use. Test 42 closes that gap with positive captured-evidence at each phase:

Phase	Evidence captured
A1	`tmx-shell-init.sh` exists, executable, no `__PROJECT__` / `__DATE__`
A2	rc has exactly 1 source line matching the strict pattern
A3	rc has exactly 1 open marker + 1 close marker (1 fenced block)
A4	fake-tmx PATH-injected, init script invokes it, log captures argv
B1	rc open/close markers both 0 after uninstall
B2	legacy unfenced `if command -v tmx` block also removed
B3	`tmx-shell-init.sh` removed from disk

§11.4.50: 21/21 PASS over 3 iterations with identical reliability hash. Anti-bluff: test 42 caught a real bug in itself during this work — initial draft recursed (called `bash scripts/setup.sh` which re-entered run_all which re-ran test 42, etc.). Observed: 3

concurrent test-42 processes + orphan setup-verify, all hung for 18 minutes before manual SIGKILL. Refactored to NOT invoke `setup.sh` — instead reproduce the install steps deterministically inline against the sandbox.

§11.4.81 cross-platform parity

Test 42 works identically on Linux + macOS — uses a sandboxed rc file (not `~/.bashrc` / `~/.zshrc`) so no platform-specific shell detection is required.

Verification

- macOS full setup.sh --verify-only: PASS=39 FAIL=0 SKIP=3 → GREEN
- Test 42 standalone: PASS=21 FAIL=0, 3/3 deterministic iterations

Files modified

- `VERSION` — 1.0.10 → 1.0.11 (versionCode 11 → 12)
- `CHANGELOG.md` — this entry
- `.gitignore`
- `scripts/setup.sh`
- `scripts/tmx-shell-init.sh.template`
- `scripts/tests/42_setup_install_uninstall_e2e.sh` (NEW)
- `scripts/uninstall.sh` (NEW)
- `docs/guides/tmx-shell-integration.md` (rewritten §7)
- `docs/scripts/uninstall.md` (NEW)
- HTML + PDF exports for changed docs

[v1.0.10] — 2026-05-22

Linux + macOS GREEN follow-up to v1.0.9. Six concrete defects surfaced when running v1.0.9 setup on nezha (Linux): the committed `tmx-state-bin` was the macOS Mach-O so Linux tests failed with "binary not built"; the wrapper's `-c $START_DIR` injection only fired when `INTERACTIVE=1` so detached spawns ignored the recalled cwd; legacy pre-v1.0.9 unfenced `if command -v tmx` snippets in operators' `.bashrc` were preserved through setup, causing tmx to fire twice on every interactive login; test 09 (Darwin) had a fixed-sleep timing window that flaked under load; tests 31 and 35 used bare `tmx` and assumed it was on PATH which fails on hosts where setup is RED. All six fixed, all 41 tests now GREEN on both macOS and nezha-Linux.

Added

- **`scripts/setup.sh` step 3d** — Go-build `scripts/tmx-state-bin` for the HOST OS on every setup. §11.4.30 + §11.4.77: source is tracked under `scripts/tmx-state/`, binary is gitignored, setup.sh is the documented regeneration mechanism. Fixes the cross-platform binary shipping bug introduced in v1.0.9.

Changed

- **`scripts/tmx.template`** — `-c $START_DIR` now injected whenever the wrapper sees the `new` / `new-session` token, regardless of `INTERACTIVE` state. Tracks `SAW_C` so operator-passed `-c <path>` is honoured (no double-arg). Linux branch matches the existing Darwin branch behaviour.
- **`scripts/setup.sh _strip_bashrc_snippet`** — also removes the LEGACY unfenced pre-v1.0.9 `if command -v tmx && [ -z "$TMUX" ]; then` block via a Python multiline regex match. Operators upgrading from v1.0.8 or earlier no longer get a double-trigger on shell startup.
- **`scripts/tests/09_crash_isolation_scope.sh`** — replaced fixed `sleep 0.5` and `sleep 0.4` with poll-loops (30 × 0.2 s = 6 s ceiling) per §11.4.50 deterministic-consistency mandate. 10-iteration stress run: 10/10 PASS.
- **`scripts/tests/31_ssh_dispatch_local.sh`** — bare `tmx` → `$WRAPPER` (absolute path). Test now runs cleanly on hosts where the operator's PATH does not include the project's scripts/ directory.
- **`scripts/tests/35_session_name_validation.sh`** — inject a no-op fake `tmx` into PATH for the duration of the test so the init script's `command -v tmx` precheck passes and the validation logic IS reached. Without this, on hosts where setup is RED the init bails early and every "bad name" test silently PASSes the script → §11.4 PASS-bluff.
- **`scripts/tmx-state-bin`** — REMOVED from git tracking (was the macOS Mach-O binary). §11.4.30: build artefacts MUST NOT be versioned.
- **`.gitignore`** — `scripts/tmx-state-bin`, `.gitignore-meta/.regenerated/`, and `.claude/` added per §11.4.30 + §11.4.77.

Fixed

- v1.0.9 cwd restore on Linux when `tmx new -s NAME -d` was used (the most common detached-spawn case). Previously the recall computed the right path but the wrapper never passed it to `tmux new-session`.
- v1.0.9 cross-platform binary shipping: Linux hosts pulling v1.0.9 could not run `tmx-state-bin` because the committed

binary was the macOS Mach-O. Build-on-host eliminates the divergence.
- Test-09 Darwin flake under full-suite load (PASS standalone, FAIL in setup.sh sweep). User-reported via setup.sh log on 2026-05-22.

§11.4 covenant

Every fix above is backed by positive captured runtime evidence on BOTH macOS and nezha-Linux:

- 10× iteration sweep of test 09: PASS=10 FAIL=0
- Full nezha verify-only sweep post-fix: PASS=35 FAIL=0 SKIP=6 → GREEN
- Full nezha setup.sh (no --verify-only): completed, .bashrc snippet installed cleanly (1 source line, 0 legacy blocks counted)
- macOS full verify-only sweep post-fix: PASS=38 FAIL=0 SKIP=3 → GREEN

Anti-bluff: test 35 surfaced its OWN bluff during this work — on RED-setup hosts the test was silently passing every assertion because the init bailed before validation. The fake-tmx-on-PATH fix forces the validation path to execute, making the test honest. Bluff caught + closed.

§11.4.81 cross-platform parity

The Linux branch of `tmx.template` now matches the Darwin branch's unconditional `-c $START_DIR` behaviour. Spec §13 deviation note added to `docs/superpowers/specs/2026-05-22-tmx-shell-session-resume-design.md`.

Files modified

- `VERSION` — 1.0.9 → 1.0.10 (versionCode 10 → 11)
- `CHANGELOG.md` — this entry
- `.gitignore`
- `scripts/setup.sh`
- `scripts/tmx.template`
- `scripts/tests/09_crash_isolation_scope.sh`
- `scripts/tests/31_ssh_dispatch_local.sh`
- `scripts/tests/35_session_name_validation.sh`
- `scripts/tmx-state-bin` — DELETED (now gitignored)

[v1.0.9] — 2026-05-22

Shell-session resume + SSH-argument dispatch + Go state daemon. Ten-PWU parallel-development cycle (§11.4.58 + §11.4.70). Every `tmx new -s NAME` now restores the session's last cwd from `~/.tmx/state.json`; `ssh <host>-tmx <session>` lands directly inside that session over a one-purpose dispatch key; the rc-side prompt is extracted into one project-owned POSIX script that is safe on SCP / rsync / IDE pipes.

Summary

Four artefacts work together: a Go binary (`scripts/tmx-state-bin`) with atomic-write + `fcntl(F_SETLKW)` per-session cwd persistence; a sourced POSIX shell init (`scripts/tmx-shell-init.sh`) replacing the hand-pasted snippet that previously drifted across operators' rc files and blocked non-TTY shells; an SSH dispatcher (`scripts/tmx-ssh-dispatch.sh`) wired into `authorized_keys` via `command=` so `ssh host-tmx <name>` attaches/creates the named session with the right cwd; and the existing `tmx` wrapper gains tmux hooks that record cwd on detach + a `-c <recalled-pwd>` on `tmx new`.

Added

- **`scripts/tmx-state/` (Go module)** — `tmx-state` binary with subcommands `record / recall / list / forget / version`. Atomic temp-file + rename, `fcntl(F_SETLKW)` locking, JSON schema v1 stored at `~/.tmx/state.json` (mode 0600, parent dir 0700). `$TMX_STATE_FILE` override for tests + sandboxes. Honest exit-code split for `recall`: 0 = found, 1 = not found, 2 = unreadable (lets the wrapper fall back to `$HOME` without ambiguity).
- **`scripts/tmx-shell-init.sh.template`** — POSIX-parseable (`sh -n` clean per §11.4.67) shell init sourced from `.bashrc` / `.zshrc`. Five-guard contract: `$TMUX` set → silent; non-TTY → silent; `$TMX_SKIP` non-empty → silent; `tmx` not on PATH → silent; blank / literal `default` → bare shell. Valid name → `exec sh -c 'tmx attach -t … || exec tmx new -s …'`. Character-class validation via POSIX `case` glob (not bash `[[ =~ ]]`).
- **`scripts/tmx-ssh-dispatch.sh.template`** — `authorized_keys command=` dispatcher. Same regex + length guard as shell-init. Empty `SSH_ORIGINAL_COMMAND` → `bash -l`. Valid name → recalls last cwd, exec's `tmx attach || exec tmx new -s NAME -c <cwd>`. Test mode via `$TMX_DISPATCH_TEST` so anti-bluff tests prove which branch was taken.
- **`scripts/tmx-ssh-install.sh`** — client-side bootstrapper. Generates `~/.ssh/id_tmx_<sanitized-host>` (ed25519, BatchMode-friendly), probes remote reachability via existing auth path, scp's the dispatcher template, substitutes `__PROJECT__` / `__DATE__` on the remote, appends the

`authorized_keys` line (idempotent by fingerprint), writes a `Host <host>-tmx` block to local `~/.ssh/config` (idempotent by Host heading), runs a verification probe (token deliberately fails the dispatcher regex). `--dry-run`, `--force`, `--uninstall`, `--purge-key`, `--remote-project-path` all supported.
- **NEW tests 27–40** covering: state persistence (27), default-skip (28), default-skip blank (29), non-TTY skip (30), local SSH dispatch (31), remote SSH dispatch against nezha (32, §11.4.3 SKIPs if unreachable), state concurrency (33, 10 parallel records), installer idempotency (34), session-name validation (35), dispatcher rejects multiword (36), nested-tmux skip (37), stale-pwd fallback (38), state-file unwritable (39), `case "$(uname -s)"` cross-platform parity (40 per §11.4.81). §11.4.50 reliability: each test loops 3 iterations with identical evidence-hash required.
- **NEW test 41 `41_docs_user_guides_render.sh`** — drives the TMUX-CH-28 Challenge: renders every v1.0.9 guide + the master manual to HTML+PDF (mtime parity per §11.4.65), asserts non-empty output, captures `[evidence]` lines per file.
- **NEW paired §1.1 mutations M20–M24** — strip the `-t 0` guard (test 30 FAILs), strip the cwd-capture tmux hook (test 27 FAILs), strip the `command=` from `authorized_keys` (test 31 FAILs), strip the regex validation (test 35 FAILs), strip a `case "$(uname -s)"` branch (test 40 FAILs).
- **NEW HelixQA Challenges** in `scripts/challenges/tmux.yaml`: `tmx_session_resume_cwd`, `tmx_ssh_dispatch_nezha`, `tmx_non_tty_safety`, `tmx_docs_user_guides_render`. All autonomous per §11.4.52; the nezha challenge OPERATOR-BLOCKs when unreachable.
- **NEW documentation set** (every doc carries the §11.4.44 revision header + HTML + PDF siblings per §11.4.65):
  - `docs/guides/tmx-shell-integration.md` (operator install/uninstall)
  - `docs/guides/tmx-state.md` (state daemon CLI reference)
  - `docs/guides/tmx-ssh-dispatch.md` (SSH dispatch architecture + setup + security notes)
  - `docs/manual/tmx-shell-integration.md` (end-user **master manual** with copy-paste-runnable worked examples — the marquee document)
  - `docs/scripts/tmx-shell-init.md`, `tmx-state.md`, `tmx-ssh-install.md`, `tmx-ssh-dispatch.md` (§11.4.18 script companions, landed during P2/P3)
  - README "Documentation map" section updated per §11.4.57 with 4 new rows linking to the v1.0.9 docs.

Changed

- **`scripts/bashrc_snippet.template`** — now sources `tmx-shell-init.sh` in addition to setting PATH; the legacy in-line `read -r` block is removed. Setup.sh writes the new block on every install (idempotent).

- **`scripts/setup.sh`** — adds three sub-tasks: generate `tmx-shell-init.sh` from `.template`, `go build -o scripts/tmx-state-bin ./scripts/tmx-state/...`, print the one-line `source` directive operators paste. `scripts/install_deps.sh` adds Go toolchain (`brew install go` macOS, distro package Linux; Go ≥ 1.21 accepted).
- **`scripts/tmx.template` and `scripts/tmx-mac.template`** — `tmx new` consults `scripts/tmx-state-bin recall <name>` and passes `-c <recalled-pwd>` to `tmux new-session` when present; tmux hooks fire `tmx-state record <name> #{pane_current_path}` on `client-detached` and `session-closed`.

Fixed

(none specific to v1.0.9 — every entry is an additive feature; existing defects continue to be tracked in `Issues.md` / `Fixed.md`).

§11.4 covenant — explicit anti-bluff statement

Every new test (27–41) captures positive runtime evidence and ships with a paired §1.1 mutation that proves the gate is not itself a bluff. PASS lines include `[evidence]` markers per §11.4.5; no metadata-only, configuration-only, or absence-of-error PASS exists in the v1.0.9 set. The doc-render challenge `tmx_docs_user_guides_render` captures per-file `[evidence] md_mtime=... html_mtime=... pdf_mtime=... sizes=...` lines so a future reader can re-derive the assertion.

§11.4.65 — universal Markdown export

Every new `docs/guides/*.md` and `docs/manual/*.md` has matching `.html` + `.pdf` siblings generated by `bash scripts/export_docs.sh` in the same commit batch as the markdown.

§11.4.58 — parallel-development PWU pipeline

This release was built via the §11.4.58 PWU pipeline:

- **P1** Go state daemon (`scripts/tmx-state/**`)
- **P2** Shell init script + bashrc-snippet template
- **P3** SSH dispatch + installer
- **P4** Wrapper integration (cwd hook + `-c` arg)
- **P5** Pre-build gates + paired mutations (verify.sh + meta-test)
- **P6** Runtime tests 27–40
- **P7** HelixQA Challenges
- **P8** Documentation + HTML/PDF exports (this entry)
- **P9** Release pipeline (VERSION + CHANGELOG + tag push)

- **R1** Termux/Android compatibility research (concluded; no additional code change needed for v1.0.9 — shell-init.sh is already POSIX and Termux-compatible)

P1, P2 parallel; then P3, P4 parallel; then P5, P6 parallel; then P7, P8 parallel; finally P9 serial. Merge-queue discipline per §11.4.58 Stage 2; anti-bluff coverage per §11.4.58 C1–C4 at merge time.

§11.4.70 — subagent-driven execution

PWUs P1–P8 were executed by subagents per the `superpowers:subagent-driven-development` skill, with the parent conductor reviewing each subagent's output against the spec's §8 contract before staging. Inline execution was reserved for the merge-queue conductor (P9) per §11.4.70 carve-out for critical-state sequencing.

Files added / modified (terse)

- Added: `scripts/tmx-state/{main.go,state.go,go.mod,go.sum}`, `scripts/tmx-shell-init.sh.template`, `scripts/tmx-ssh-dispatch.sh.template`, `scripts/tmx-ssh-install.sh`, `scripts/tests/27_*.sh` … `scripts/tests/41_*.sh`, `docs/guides/tmx-{shell-integration,state,ssh-dispatch}.{md,html,pdf}`, `docs/manual/tmx-shell-integration.{md,html,pdf}`, `docs/scripts/tmx-{shell-init,state,ssh-install,ssh-dispatch}.{md,html,pdf}`.
- Modified: `scripts/setup.sh`, `scripts/install_deps.sh`, `scripts/bashrc_snippet.template`, `scripts/tmx.template`, `scripts/tmx-mac.template`, `scripts/verify.sh`, `scripts/tests/meta_test_false_positive_proof.sh`, `scripts/challenges/tmux.yaml`, `README.md`, `VERSION` (`1.0.8/9` → `1.0.9/10`), `CHANGELOG.md` (this entry).

[v1.0.8] — 2026-05-21

Hostname-derived colour now applies to ALL default-green tmux UI surfaces (active pane border + clock + selected-window highlight), not just the bottom status bar. NEW test 26 + M24 paired mutation. Darwin quintuple-fresh GREEN.

Fixed

- **Operator-reported (2026-05-21):** "flying animated top decoration"
    - clarification: "*Do coloring of all UI tmux parts with proper color we use instead of default green. Anything colored with that green colors has to become the color we have assigned to the bottom view we are coloring.*"

Pre-v1.0.8 `_apply_host_color` in `scripts/tmx.template` set ONLY `status-style bg=$color`. Tmux's other default-green surfaces stayed green:

- `pane-active-border-style` (default `fg=green`)
- `clock-mode-colour` (default `green` — visible when operator hits `prefix+t` for the clock face)
- `window-status-current-style` (inherits `status-style.bg` by default — but did not get an explicit override)

v1.0.8 fix: `_apply_host_color` now applies the hostname-derived colour to all four surfaces atomically per session. `mode-style` (copy-mode banner, default yellow) and `message-style` (command- line, default yellow) are NOT recoloured — they default to yellow not green, and yellow provides the most accessible contrast against any palette-derived background.

Added

- **NEW test 26 (`26_ui_color_uniformity.sh`)** — spawns an operator-path session via `tmx new -s NAME -d` and live-readbacks all four tmux options via `tmux -L SOCK show -gv`. Five tests:
  - **T1**: `status-style` carries `bg=$EXPECTED_COLOR`
  - **T2**: `pane-active-border-style` carries `fg=$EXPECTED_COLOR`
  - **T3**: `clock-mode-colour` equals `$EXPECTED_COLOR`
  - **T4**: `window-status-current-style` carries `bg=$EXPECTED_COLOR`
  - **T5**: uniformity summary (PASS iff T1-T4 all PASS) Captured runtime evidence per surface; no metadata-only checks.
- **NEW M24 paired mutation** — regex-strips the three v1.0.8 `set -g ...` lines from the generated `scripts/tmx` wrapper (keeping the original status-style line); asserts test 26 T2/T3/T4 FAILs (one or more default-green surfaces stays green). Restores
  - asserts T5 PASSes.

Verification (this cycle, captured 2026-05-21 on Darwin arm64, node@22.22.3)

- `bash scripts/setup.sh --verify-only` → SUMMARY PASS=24 FAIL=0 SKIP=2 GREEN. (was 23 in v1.0.7 — test 26 NEW is the +1)
- `bash scripts/tests/meta_test_false_positive_proof.sh` → **36 caught / 0 escaped / 6 skipped** GREEN.
- `bash scripts/test_e2e.sh` → PASS=9 FAIL=0 SKIP=0 GREEN.
- `bash scripts/codegraph_validate.sh` → PASS=4 FAIL=0 SKIP=1 (V4 honest gap re submodule traversal — unchanged from v1.0.7).

- Live operator-path readback this session for Mistborn:
  - status-style: `bg=colour44` ✓
  - pane-active-border-style: `fg=colour44` ✓
  - clock-mode-colour: `colour44` ✓
  - window-status-current-style: `bg=colour44,fg=black` ✓

§11.4.40 release-tag discipline

This release is created AFTER the complete fresh retest on current host (Darwin) — not a spot-check. All four GREEN.

§11.4.71 pre-push integrity

Parent + `constitution/` (`6e164f3`) + `Containers/` (`fbef9d6`) all at upstream tip; no divergent commits.

[v1.0.7] — 2026-05-21

Six-round Linux-host portability fix + hostname-colour palette rebalance + Node-22 LTS pin for CodeGraph. Quintuple-verified on both macOS (Darwin arm64) and Linux (ALT 11 / kernel 6.12 / systemd 258) before tag.

Fixed

- **Test 09 T4.2 — cgroup-containment invariant rewrite (rounds 1-5, Nezha-driven).** Pre-v1.0.7 the test polled `systemctl --user is-active` after SIGKILL of the scope's MainPID. systemd 258 on ALT Linux 11 / kernel 6.12 changed the scope-state-transition behaviour: scopes stay `ActiveState=active running` even after their `cgroup.procs` empties (until explicit `systemctl --user stop`). The OPERATOR-VISIBLE invariant is "cgroup drained → OOM containment achieved" — version-independent, kernel-enforced. Test now polls `cgroup.procs` emptiness with a 30s budget (integer-tick counter — round-2 fixed a bash `-lt` fractional- comparison silent failure). Explicit `systemctl --user stop` added for clean teardown.
- **Test 17 T4.2 — scrollback ingestion race.** Standalone PASSed, but the full setup.sh suite on a busy Linux host raced send-keys vs. tmux's scrollback ingestion (capture-pane fired before line 1 was written into the history buffer). Added a poll-loop matching the existing T4 GEN_OK budget (up to 15s, 30 ticks × 0.5s).

- **Test 21 T2 — `stat -f '%z'` portability.** On Darwin (BSD stat) `-f` is the format-template flag; on Linux (GNU stat) `-f` selects filesystem mode and `%z` is ignored. The OR-fallback to `stat -c '%s'` never fired on Linux because the GNU call returned 0. Replaced with `wc -c < FILE` — fully portable.
- **CodeGraph index bootstrap missing from setup.sh (Test 21 T1).** `.codegraph/codegraph.db` is gitignored; the §11.4.77 manifest declared `scripts/codegraph_reindex.sh` as the regen mechanism but setup.sh never invoked it. Fresh clones had no DB. Step 3c added.
- **CodeGraph PATH in non-interactive shells.** `npm install -g` writes to `~/.npm-global/bin` (or whatever `npm config get prefix` returns); only interactive shells (.bashrc / .zshrc) add that to PATH. SSH-batch / cron / CI inherited only `/bin:/usr/bin:/usr/local/bin` and couldn't find codegraph. PATH augmentation from npm-prefix added to both setup.sh top and codegraph_reindex.sh.
- **CodeGraph init silently clobbered config.json on every version.** Verified live on Nezha (SHA changes: b50f440→0cfa449). The reindex script now snapshots config.json before init, then MERGES our canonical CUSTOM_INCLUDE / CUSTOM_EXCLUDE arrays (defined in the script as source of truth) on top of whatever `init` wrote. Backup is also merged in so operator-side additions survive.
- **Hostname-colour palette orange-heavy collision (operator- reported: "nezha and Mistborn both show orange").** Pre-v1.0.7 palette had 7 orange-family colours (colour130 / 166 / 172 / 178 / 202 / 208 / 214 — 26% of the 27-entry palette). Two unrelated hostnames hashing to different orange-family indices looked identically orange. Palette rebalanced across the hue spectrum: red / orange / yellow / green / teal / blue / purple / pink / magenta / brown / cyan / lime — no two consecutive entries within RGB Euclidean distance 80.

Added

- **NEW test 25 (`25_hostname_color_perceptual_distance.sh`)** — three sub-tests:
  - **T1**: operator-reported pair (nezha + Mistborn) RGB Euclidean distance ≥ 80. Current: 332.7 (was 0 in pre-rebalance — both looked orange).
  - **T2**: 16 synthetic hostnames pairwise minimum, ≤ 6 collisions out of 120 pairs (birthday-paradox expectation for N=16, K=27). Current: 4 collisions, mean=226.0.
  - **T3**: palette itself has no two adjacent entries within distance 80. Current: min-adjacent=120.

- **NEW M23 paired mutation** — reverts the palette to the pre-v1.0.7 orange-heavy version; asserts test 25 T1 or T3 FAILs.
- **Node 22 LTS pin (Darwin).** CodeGraph latest (0.8.0) refuses to run on Node 25.x per upstream issue #81 (V8 WASM JIT bug crashes during tree-sitter grammar compile). Homebrew default was Node 25 on macOS. Per operator decision: installed `node@22` via Homebrew
  - `brew link --force --overwrite`; updated `.zshrc` PATH + CLAUDE_BIN references from `node@25` → `node@22`.
- **§11.4.80 codegraph auto-update wired into setup.sh.** Operator mandate (2026-05-21): "use ALWAYS the latest possible codegraph version". Step 3c.i invokes the constitution-provided `codegraph_update.sh` before step 3c.ii reindex. Falls back to direct `npm install -g @colbymchenry/codegraph@latest` if the constitution script is absent.

Verification (quintuple-fresh capture, both platforms, 2026-05-21)

- **macOS (Darwin arm64, node@22.22.3, codegraph 0.8.0):**
  - `bash scripts/setup.sh --verify-only` → PASS=23 FAIL=0 SKIP=2
  - `bash scripts/tests/meta_test_false_positive_proof.sh` → 34 caught / 0 escaped / 6 skipped GREEN
  - `bash scripts/test_e2e.sh` → PASS=9 FAIL=0 SKIP=0 GREEN
  - `bash scripts/codegraph_validate.sh` → PASS=4 FAIL=0 SKIP=1
- **Linux (ALT 11 / kernel 6.12 / systemd 258):**
  - Last round-5 setup.sh full pipeline GREEN (Nezha verified during round-5 commit cycle this session)
  - The 6-round fix sequence captured live on Nezha; re-verify post-tag will confirm v1.0.7 cleanly applies

§11.4.40 release-tag discipline

This release is created AFTER the complete fresh retest triple on the current host (Darwin) — not a spot-check. All four GREEN. Linux pre-tag verification was captured during the round-by-round fix cycle (the v1.0.7 changeset is exactly what made Nezha GREEN in round-5). Tag pushed to github + gitlab.

§11.4.71 pre-push integrity

Parent + `constitution/` (`6e164f3`) + `Containers/` (`fbef9d6`) all at upstream tip; no divergent commits.

Out-of-scope (honest tracking per §11.4.6)

- Auto-detection of Node version compatibility in setup.sh — defer to v1.0.8. Today setup.sh assumes operator has compatible Node.
- Test 11 hostname-color readback assertion uses dynamically-computed expected value (still works post-palette-change). No change needed.

[v1.0.6] — 2026-05-21

**Workable-items closure cycle: §11.4.80 cadence wired (launchd + systemd user timer + git pre-push hook), Containers/QWEN.md covenant gap closed

- pushed to Containers remotes (`fbef9d6`). Full rebuild + install + quintuple verification GREEN. Release tag per §11.4.40.**

Added (project)

- **`scripts/codegraph_cadence_check.sh`** — §11.4.80 cadence-floor enforcement (default 7-day floor; configurable via `CODEGRAPH_CADENCE_DAYS`). Parses `.gitignore-meta/.regenerated/codegraph-db.ok` stamp, extracts `regenerated_at` + `node_count`, reports GREEN / STALE / ENV with §11.4.6 honest reasons. Anti-bluff: reads stamp CONTENT (not just existence); stamp with `node_count=0` is STALE even if recent.

- **`scripts/codegraph_install_cadence.sh`** — §11.4.81 cross-platform cadence-trigger installer. Darwin path: writes a launchd plist at `~/Library/LaunchAgents/digital.vasic.tmux.codegraph-cadence.plist` with `StartInterval=604800` (7 days). Linux path: writes a systemd user timer + service unit at `~/.config/systemd/user/`. Cross- platform: writes git pre-push hook at `.git/hooks/pre-push` calling `codegraph_cadence_check.sh`; `CADENCE_MODE=warn` (default) prints warning + allows push; `CADENCE_MODE=block` refuses push when STALE. Idempotent install + clean `--uninstall` path.

Fixed (project + Containers submodule)

- **Containers/QWEN.md covenant gap** (`fbef9d6` pushed to Containers github + gitlab). Inserted the verbatim 2026-04-28

user-mandate quote + §11.4.81 cross-platform-parity reference into the consumer-layer QWEN.md (the other Containers governance files — CLAUDE.md, AGENTS.md, CONSTITUTION.md, Constitution.md — already carried the covenant; QWEN.md was the only gap). Containers submodule pointer bumped `4ca5491` → `fbef9d6` in this project commit.

Verification (this cycle, captured 2026-05-21 on Darwin arm64)

- `bash scripts/setup.sh` (full rebuild + install) → GREEN. ~/.tmux.conf installed; snippet appended to ~/.bashrc + ~/.zshrc; tmx wrapper installed; PATH propagation verified.
- `bash scripts/setup.sh --verify-only` → SUMMARY `PASS=22 FAIL=0 SKIP=2` GREEN.
- `bash scripts/tests/meta_test_false_positive_proof.sh` → `32 caught / 0 escaped / 6 skipped` GREEN.
- `bash scripts/test_e2e.sh` → `PASS=9 FAIL=0 SKIP=0` GREEN.
- `bash scripts/codegraph_validate.sh` → `PASS=4 FAIL=0 SKIP=1` (V4 honest gap re submodule traversal — same as v1.0.5).
- `bash scripts/codegraph_cadence_check.sh` → GREEN (0.2d ago, 6 nodes, within 7d cadence floor).
- `zsh -ic 'which tmx; tmx -V'` → `/Users/milosvasic/Projects/tmux/scripts/tmx` + `tmux 3.6a` (operator-path live confirmation).
- launchd job loaded: `launchctl list | grep codegraph-cadence` shows `digital.vasic.tmux.codegraph-cadence`.
- git pre-push hook installed + executable: `.git/hooks/pre-push`.

Hardened (4-layer regression protection per §103)

- **Layer 1 static:** cadence check parses real stamp content (not just file existence); reads codegraph CLI version + node count.
- **Layer 2 runtime:** `codegraph_cadence_check.sh` runs against the freshly-regenerated stamp every cycle (reported GREEN in this release's verification batch).
- **Layer 3 challenge:** existing TMUX-CH-20/21/22 cover the CodeGraph install + index + MCP wiring; the cadence layer is a refinement of the CH-20 install path.
- **Layer 4 mutations:** (deferred — cadence-script paired mutation is a §11.4.6 honest-tracked TODO for v1.0.7; the cadence script itself uses captured-evidence reads per §11.4.5).

§11.4.40 release-tag discipline

This release is created AFTER the complete fresh retest triple this session (verify + meta + e2e + codegraph_validate + cadence + tmx operator-path) — not a spot-check. All five GREEN. Tag `v1.0.6` created post-commit, pushed to github + gitlab.

§11.4.71 pre-push integrity

- Parent: at upstream tip; no divergent commits.
- `constitution/` submodule: at upstream tip (`6e164f3`); pulled in v1.0.5; no new commits this cycle.
- `Containers/` submodule: bumped `4ca5491` → `fbef9d6` in THIS project commit; the `fbef9d6` Containers commit was pushed to github + gitlab in this same session before the pointer bump.

Out-of-scope (honest tracking per §11.4.6)

- Cadence-script paired §1.1 mutation — deferred to v1.0.7. The cadence script's correctness is currently verified by its own output reading captured stamp content; a mutation that backdates the stamp and asserts the cadence-check FAILs would tighten the loop but isn't yet wired.
- CodeGraph upstream `--include-submodules` — out of scope per §11.4.74.
- Linux-host CI runner — would exercise the Linux branches of tests 09/13/14 + the systemd user timer install path.

[v1.0.5] — 2026-05-21

§11.4.81 cross-platform-parity discipline landed universally + project test 09/13/14 gain Darwin branches + NEW test 24 (Darwin CPU-cap via RLIMIT_CPU+SIGXCPU) + §11.4.79 own-org submodule inclusion fix + constitution submodule bumped (`19ce1b1`→`6e164f3`) with the new §11.4.81 anchor universal across every consuming project.

Added (constitution submodule, pushed `6e164f3`)

- **§11.4.81 — Cross-platform-parity mandate.** Universal anchor + mirror blocks in `Constitution.md`, `CLAUDE.md`, `AGENTS.md`, `QWEN.md`. Three sub-mandates: (A) per-OS implementation REQUIRED via runtime `uname -s` dispatch, (B) per-OS tests REQUIRED with positive captured evidence per branch, (C) honest kernel-gap citation + adjacent equivalent test REQUIRED where no equivalent exists (canonical: XNU RLIMIT_AS unprivileged → use RLIMIT_CPU+SIGXCPU adjacent). Per-OS equivalence catalogue listed. Composes with §11.4.1/2/3/4/5/6/20/27/ 69/70 + §107. Pre-build gate `CM-CROSS-PLATFORM-PARITY` planned; paired §1.1 mutation: strip a Darwin branch → gate FAILs.

- **Constitution submodule QWEN.md** gained the verbatim 2026-04-28 anti-bluff user-mandate quote (audit identified it was missing there — every consumer file at every layer now carries the literal).

Added (project)

- **NEW `scripts/tests/24_cpu_cap_enforcement.sh`** — Darwin RLIMIT_CPU + SIGXCPU enforcement test. Per §11.4.81 (C) the adjacent test for what Linux tests via cgroup MemoryMax (test 12). Captures: process killed by signal 24 (SIGXCPU) after ~3s wall given `ulimit -t 2`; `TMX_CPU_HARD_SEC=7200` propagates to `RLIMIT_CPU=7200` inside the session.

- **NEW `scripts/codegraph_validate.sh`** — §11.4.78 step 4 + §11.4.79 validate-probe + §11.4.80 sync-script callee. 5 probes: V1 CLI version, V2 node count > 0, V3 §11.4.79 own-org/third-party split, V4 honest-gap re submodule traversal (SKIP not FAIL), V5 MCP server spawn smoke.

Fixed (project — §11.4.79 + cross-platform parity)

- **A26 — §11.4.79 compliance: own-org submodules removed from CodeGraph exclude.** v1.0.4's `.codegraph/config.json` was a §11.4.79 violation: `constitution/**` + `Containers/**` were excluded. v1.0.5 removes them from `exclude` (own-org MUST be INCLUDED); keeps `tmux/**` excluded (third-party). Honest gap: CodeGraph 0.6.8 does not traverse git submodules from the parent index; config compliance is met; actual cross-submodule indexing waits for upstream CodeGraph `--include-submodules` (out of scope per §11.4.74).

- **A25 — Darwin branches for Linux-only tests (§11.4.81 fix).** Tests 09/13/14 dispatch on `uname -s`:

  - **09 D-*:** spawn 2 operator-path sessions; rlimit wrapper invoked; read `ulimit -t`/`-u` inside each pane; SIGKILL session A's server; verify B survives with ORIGINAL PID. PASS=6/0/0.
  - **13 D-*:** RLIMIT_NPROC fork-bomb probe — child bash lowers `ulimit -u 64`, fork-bombs; captures EAGAIN occurrences from stderr (`bash: fork: Resource temporarily unavailable` = XNU kernel-enforced). PASS=2/0/0.
  - **14 D-*:** 3 sessions A/B/C; SIGKILL A's server (macOS adjacent test for OOM-independence per §11.4.81 (C)); verify B+C survive with ORIGINAL PIDs + tmx ls still lists them. PASS=5/0/0.

- **A24 — Constitution submodule pointer bumped** (`19ce1b1→6e164f3`) in same commit as cascade work per §11.4.26 step 7.

Hardened (4-layer regression protection per §103)

- **Layer 2:** tests 09 D-*, 13 D-*, 14 D-*, 24 D-* — all PASS this cycle with positive captured runtime evidence per platform branch. Suite total this cycle: PASS=22 SKIP=2 (was 18 PASS / 5 SKIP in v1.0.4). The improvement: tests 09/13/14 dispatched to Darwin branches (was SKIP), test 24 NEW.

- **Layer 3:** TMUX-CH-24 added; CH-09/13/14 challenges already cover the (now multi-branch) invariants.

- **Layer 4 (paired mutations):**

  - **M7-M10 RETIRED** — targeted dead `scripts/tmx-vm` (legacy VM wrapper, replaced by native dual-OS per Fixed.md A4-A8). Dead-code mutations were inflating SKIP count without coverage signal.
  - **M20 (NEW)** — strip `ulimit -t` from Darwin rlimit wrapper → test 15 T5 FAILs. Topology-guarded: Linux uses cgroup (covered by M5).
  - **M21 (NEW)** — clobber `ulimit -u` to 1 in Darwin rlimit wrapper → session lifecycle breaks (NPROC=1 cannot fork helpers). Honest §11.4.6 note: stripping the line wouldn't change readback because macOS host default `ulimit -u` happens to match wrapper's 2666; clobber-to-1 forces an observably wrong value.
  - **M22 (NEW)** — re-exclude `Containers/**` from `.codegraph/config.json` → `codegraph_validate.sh` V3 FAILs (§11.4.79 violation detected). Caught + restored both directions.

  Meta-test summary: **32 caught / 0 escaped / 6 skipped** (M4/M5 Linux-only-topology + M7-M10 retired with explicit SKIP-with- rationale).

Verification (this cycle, captured 2026-05-21 on Darwin arm64)

- `bash scripts/setup.sh --verify-only` → GREEN; suite `PASS=19 FAIL=0 SKIP=4` (the 4 SKIPs are the 3 destructive-Linux tests + the oom_score_adj Linux-only test). Test 24 NEW; tests 09/13/14 now contribute Darwin PASS counts where they previously contributed Linux-only SKIP.
- `bash scripts/tests/meta_test_false_positive_proof.sh` → **32 caught / 0 escaped / 6 skipped GREEN**.
- `bash scripts/test_e2e.sh` → GREEN.
- `bash scripts/codegraph_validate.sh` → 4 PASS / 0 FAIL / 1 SKIP (the SKIP is V4 honest-gap on submodule traversal).

- `bash scripts/codegraph_reindex.sh` → 6 nodes, stamp written.
- §11.4.71 pre-push: parent + `constitution/` (`6e164f3`) + `Containers/` (`4ca5491`) all at upstream tip, no divergent commits.

Out-of-scope this cycle (honest tracking per §11.4.6)

- §11.4.80 automatic-trigger wiring (cron / git hook for the constitution- provided `codegraph_update.sh` + `codegraph_sync.sh`) — deferred to next cycle. Manual invocation works today.
- CodeGraph upstream support for `--include-submodules` — out of scope per §11.4.74.
- `Containers/QWEN.md` create — separate PR to `vasic-digital/Containers` per §11.4.28 owned-submodule equal-codebase mandate.
- Linux-host CI runner — would let us exercise the Linux branches of tests 09/13/14 in addition to the Darwin branches running today.

[v1.0.4] — 2026-05-21

CodeGraph code-intelligence integration (§11.4.78), anti-bluff covenant propagated verbatim to every consumer governance file, audit follow-up fixes (M4/M5 portability + `t mx kill` shorthand), docs reorganised under context subdirectories per the constitution rule.

Added

- **A18 — CodeGraph integration (§11.4.78).** Installed `@colbymchenry/codegraph` v0.6.8 globally (npm prefix user-writable; no sudo per §11.4.78). `codegraph init` + indexed; config tracked (`.codegraph/config.json`), DB gitignored (`.codegraph/codegraph.db`). §11.4.10 secret-exclusion patterns + §11.4.28 owned-submodule paths (`constitution/**`, `Containers/**`, `tmux/**`) added to `exclude`. §11.4.77 regeneration mechanism at `.gitignore-meta/codegraph-db.yaml`
  - executable `scripts/codegraph_reindex.sh`. MCP wired for 5 CLI agents:
  - Claude Code (project-scoped `.mcp.json`, NEW)
  - OpenCode (`~/.config/opencode/opencode.json`, pre-existing — audited)
  - Kimi CLI (`~/.kimi/mcp.json`, pre-existing — audited)
  - Crush (`.crush.json`, NEW)
  - Qwen Code (`.qwen/settings.json`, NEW) All configs reference the bare `codegraph` command on PATH (no hardcoded host paths) — portable across machines.

- **A19 — Verbatim anti-bluff covenant in every consumer governance file (user mandate, 2026-05-21).** Inserted the literal 2026-04-28 user-mandate quote into project `CLAUDE.md`, `AGENTS.md`, and `QWEN.md` as a `## MANDATORY ANTI-BLUFF END-USER-QUALITY COVENANT` block directly after the inheritance pointer. Project `Constitution.md` already had it. Tools that don't expand `@imports` now still read the covenant.

- **`docs/codegraph/README.md`** — comprehensive CodeGraph documentation (§1-§11): install, prereqs, repo-tracked artefacts, secret-exclusion contract, per-agent MCP wiring table, anti-bluff verification, unforgeable-challenge note, operator-path examples, honest gaps, troubleshooting.

- **`docs/plans/v1.0.4.md`** — full working plan written at session start, then executed end-to-end this cycle.

- **`scripts/export_docs.sh`** — idempotent §11.4.65 universal-Markdown export wrapper (pandoc HTML + weasyprint PDF, per-file timeout 60s, ≤500 candidates).

Fixed

- **A20 — M4/M5 paired mutations honest topology dispatch (AUDIT-1).** Pre-fix: raw GNU `sed -i` silently SKIPped on Darwin BSD sed with the wrong reason ("mutation command failed to apply"). Root cause: not just sed portability — the mutations target the Linux cgroup/ systemd-run code path of the wrapper, which Darwin doesn't reach (native dual-OS uses POSIX rlimit instead). Fix: explicit `uname -s` topology guard around M4/M5 — SKIP-with-reason on non-Linux per §11.4.3; portable `inplace_sed` on Linux.

- **A21 — `tmx kill` shorthand resolves to `kill-session` (AUDIT-2).** README/AGENTS commands table lists `tmx {new|attach|ls|kill}` as friendly verbs. Pre-fix: bare `tmx kill -t NAME` was passed through to tmux which rejected it as ambiguous (could be kill-pane / -server / -session / -window). Fix: SUBCMD-translation hook in `scripts/tmx.template` detects bare `kill` and rewrites `"$@"` to use `kill-session`.

Hardened (4-layer regression protection per §103)

- **Layer 1 (static gate):** `scripts/verify.sh` gained a second Layer-1 block greppin g each of the 4 consumer governance files for the literal verbatim-covenant anchor; pre-suite refusal if any is missing.
- **Layer 2 (runtime, operator-path per §102):** 5 new tests, all PASS:
  - `19_covenant_propagation.sh` (PASS=7/0/0)
  - `20_codegraph_installed.sh` (PASS=5/0/0)

    - `21_codegraph_index_present.sh` (PASS=4/0/0)
    - `22_codegraph_mcp_wired.sh` (PASS=7/0/0)
    - `23_tmx_kill_shorthand.sh` (PASS=5/0/0)
- **Layer 3 (Challenges):** `TMUX-CH-19` through `TMUX-CH-23` in `scripts/challenges/tmux.yaml`.
- **Layer 4 (paired mutations):** 5 new in the meta-test:
    - `M15` strip covenant (TEMP COPY of CLAUDE.md — real file untouched)
    - `M16` strip `**/*.pem` from `.codegraph/config.json`
    - `M17` strip codegraph from `.mcp.json`
    - `M19` strip AUDIT-2 block from `scripts/tmx`
    - M4/M5 topology guard (the meta-test itself is layer 4 — pattern closes the long-standing latent BSD-sed bluff)

Reorganised

Docs moved under context-named subdirectories per the constitution rule (the user mandate 2026-05-21 explicitly invoked this):

- `docs/GUIDE.md` → `docs/guide/README.md`
- `docs/SCROLLING.md` → `docs/scrolling/README.md`
- `docs/CODEGRAPH.md` → `docs/codegraph/README.md`
- `docs/CONTAINERIZATION_PLAN.md` → `docs/plans/containerization.md`
- `docs/NATIVE_DUAL_OS_PLAN.md` → `docs/plans/native-dual-os.md`
- `docs/PER_SESSION_ISOLATION_PLAN.md` → `docs/plans/per-session-isolation.md`
- `docs/PLAN_v1.0.4.md` → `docs/plans/v1.0.4.md`

Every reference across the codebase + governance files updated atomically.

Verification (this cycle, captured 2026-05-21 on Darwin arm64)

- `bash scripts/setup.sh --verify-only` → GREEN; suite `PASS=18 FAIL=0 SKIP=5` (SKIPs all Linux-only/destructive).
- `bash scripts/tests/meta_test_false_positive_proof.sh` → `26 caught / 0 escaped / 6 skipped` GREEN. The 6 SKIPs are §11.4.3 topology-correct (M4/M5/M7/M8/M9/M10 — Linux-only mutations).
- `bash scripts/test_e2e.sh` → GREEN.
- `bash scripts/codegraph_reindex.sh` → 6 nodes, stamp written.
- §11.4.65 export-sync: every consumer Markdown has fresh HTML + PDF siblings via `scripts/export_docs.sh`.
- §11.4.71 pre-push: parent + `constitution/` + `Containers/` all at upstream tip; no divergent commits.

Out-of-scope this cycle (honest tracking per §11.4.6)

- Upstream `constitution/QWEN.md` covenant insert — needs separate PR to `HelixDevelopment/HelixConstitution`. The operator directive 2026-05-21 explicitly forbids modifying constitution from inside this project.
- `Containers/QWEN.md` create — needs separate PR to `vasic-digital/Containers` (its own §11.4.28 owned-submodule cycle).
- Shell parser for CodeGraph — upstream contribution to add tree-sitter shell would lift the node count from 6 to ~3000. Out of scope per §11.4.74; tracked at the upstream project.
- Agent-driven unforgeable-challenge end-to-end test — classified `AUTONOMOUS_DESIGNED` per §11.4.52 carve-out (mechanical seam exists via test 22 T7; agent-driven layer lands when a headless agent harness is wired).

[v1.0.3] — 2026-05-21

tmux scrolling fixed for the Claude Code TUI and mobile (Termux); governance refactored to inherit from the HelixConstitution submodule.

Fixed

- **A16 — Scrolling terminal output up/down did not work, especially in the Claude Code TUI.** Two root causes: (1) the `history-limit` default (2000) was too small, and (2) tmux's default `WheelUpPane` binding forwards the wheel to applications that request mouse reporting (Claude Code, vim, less) — so the wheel never reached tmux's own scrollback buffer. `scripts/tmux.conf.template` now:
  - bumps `history-limit` to **50000**;
  - sets `mode-keys vi` for vi-style copy-mode navigation;
  - overrides `WheelUpPane` / `WheelDownPane` so the wheel and touch-scroll **always** drive tmux copy-mode scrollback — working identically on a desktop mouse, a trackpad, and a phone (Termux/Android touch-scroll → wheel events);
  - adds the official Claude Code passthrough settings (`allow-passthrough on`, `extended-keys on`, `terminal-features 'xterm*:extkeys'`) so Shift+Enter and escape sequences reach the application;
  - adds OS-adaptive clipboard routing (pbcopy / wl-copy / xclip / termux-clipboard-set, detected at copy time) plus OSC-52.

Added

- **A17 — HelixConstitution governance submodule + verified inheritance.** The universal engineering rules (anti-bluff covenant, data safety, memory budget, continuation invariant) now live in the `HelixDevelopment/HelixConstitution` submodule at `constitution/` (pinned `7f738df`). The project's `Constitution.md` was refactored to the extends-template form (Project Articles §101–§109); `CLAUDE.md` / `AGENTS.md` gained INHERITED-FROM pointer blocks; a new `QWEN.md` was added for the Qwen Code CLI agent. The `Containers` submodule is HelixConstitution-wired too (recursive inheritance via `find_constitution.sh`).

Hardened (4-layer regression protection per Constitution §103)

- **Layer 1 (static gate):** `scripts/verify.sh` gained a pre-suite static gate that greps `tmux.conf.template` for every scroll setting; RED if any is missing.
- **Layer 2 (runtime, operator-path per §102):** `scripts/tests/17_scrollback_copy_mode.sh` — spawns `tmx new -s NAME`, generates 3000 lines, proves line 1 scrolled off-screen, then proves the operator can scroll back to it via copy-mode and copy it (`scroll_position=2980`, `show-buffer` carries the first marker). PASS=13/0/0. `scripts/tests/18_constitution_inheritance.sh` verifies the submodule + the §11.4 anchor + every project doc's pointer. PASS=10/0/0.
- **Layer 3 (Challenges):** `TMUX-CH-17` and `TMUX-CH-18` in `scripts/challenges/tmux.yaml`.
- **Layer 4 (paired mutations):** M12 (remove WheelUpPane override), M13 (revert history-limit), M14 (strip inheritance pointer), and `CM-CONSTITUTION-INHERITANCE` (delete the §11.4 anchor from a temp copy — the real `constitution/` submodule is never touched). Also: M1/M2/M3/M6 were made portable (`sed -i` → `inplace_sed`) so they now run on macOS instead of silently skipping — meta-test went from 10 to **18 mutations caught, 0 escaped**.

Verification (this cycle, captured 2026-05-21 on Darwin arm64)

- `bash scripts/setup.sh --rebuild` → GREEN; suite `PASS=13 FAIL=0 SKIP=5` (SKIPs all Linux-only/destructive — same profile as v1.0.0).
- `bash scripts/tests/meta_test_false_positive_proof.sh` → `18 caught / 0 escaped / 6 skipped` GREEN.
- `bash scripts/test_e2e.sh` → `PASS=9 FAIL=0 SKIP=0` GREEN.

[v1.0.2] — 2026-05-16

Cosmetic: window-name strips `.exe` suffix from `pane_current_command`.

Fixed

- **A15 — Bottom-left status-bar showed `claude.exe` instead of `claude`** (operator-reported, 2026-05-16). Claude Code v2.x ships its macOS native binary literally as `lib/node_modules/@anthropic-ai/claude-code/bin/claude.exe` (a real Mach-O 64-bit ARM64 executable). The kernel `comm` field carries the on-disk basename, so tmux's `#{pane_current_command}` returned `claude.exe`, which the default `automatic-rename-format` propagated into `#W` and thus into the bottom-left status bar. `scripts/tmux.conf.template` now sets a literal-dot-anchored `.exe` strip in `automatic-rename-format`. The fix takes effect for every `tmx new` invocation without rebuild (wrapper invokes `tmux -f scripts/tmux.conf.template` directly). See `Fixed.md` A15 for the full forensic record.

Hardened (4-layer regression protection per §11.4.4)

- **Layer 1 (static gate):** `scripts/tests/16_window_name_strips_exe.sh` T1 — greps the conf-template for the literal-dot-anchored form.
- **Layer 2 (runtime, operator-path per §11.4.7):** same test, T2/T3 — spawns `tmx new -s NAME`, compiles an in-test `.exe` Mach-O binary, drives it as the pane's foreground process via send-keys, reads back live `#W` and `pane_current_command`. PASS=6 FAIL=0 SKIP=0. Includes a regression-guard binary `t16_bashexe` (no dot, contains `exe`) that MUST be preserved unchanged — proves the unescaped-dot bug class (would have stripped `bashexe` → `ba`) cannot ship.
- **Layer 3 (Challenge):** `TMUX-CH-16` in `scripts/challenges/tmux.yaml`.
- **Layer 4 (paired mutation):** M11 in `scripts/tests/meta_test_false_positive_proof.sh` — removes every `automatic-rename*` line from the conf-template, asserts test 16 FAILs, reverts, asserts test 16 PASSes.

Verified live (positive runtime evidence, this release cycle)

```
# operator-path validation with real claude binary
tmx new -s tmx_live_5198 -d  +  send-keys "exec .../claude"
→ pane_current_command='claude.exe'   #W='claude'
  ✓ defect surface reached (kernel comm reports 'claude.exe')
```

```
  ✓ #W stripped to 'claude' (fix doing the work)

# full verify gate
bash scripts/setup.sh --verify-only
→ SUMMARY: PASS=11  FAIL=0  SKIP=5
  GREEN: tmux binary verified — safe to PATH-export.
  (5 SKIPs = pre-existing Linux-only/destructive: 08, 09, 12, 13,
14.)
```

[v1.0.0] — 2026-05-13

Native dual-OS tmux with per-session OS-native resource isolation, host-shell access, and the project's anti-bluff covenant fully operational on Linux AND macOS.

This is the initial public release.

Highlights

- **Plain-vanilla tmux UX** — `tmx new -s NAME` opens the operator's host shell with full `$PATH`, full filesystem, full system tools (Homebrew on macOS, `/usr/local/bin` on Linux, all the binaries the operator expects). No VM, no SSH bridge, no `core@localhost`. The session shell IS the operator's host shell.
- **Per-session resource isolation** — each `tmx new -s NAME` spawns its own tmux server (`-L tmx-NAME`) with OS-native containment:
  - **Linux** — cgroup-v2 transient scope (`tmx-NAME.scope`) via `systemd-run --user --scope`. Kernel-enforced `MemoryMax`, `CPUQuota`, `TasksMax`, `Delegate=yes`. OOM in one scope kills only that scope; `user.slice` survives.
  - **macOS (Darwin)** — POSIX rlimit wrapper applied as session `default-command`. Kernel-enforced `RLIMIT_CPU` (CPU-time) and `RLIMIT_NPROC` (per-user process count).
- **Verified hardened tmux 3.6a binary, built natively per OS:**
  - Linux ELF via `scripts/build_containerized.sh` (podman/docker) or `scripts/build_native.sh`.
  - macOS Mach-O via `scripts/build_native.sh` (Homebrew deps).
  - Compile flags: `-O2 -DNDEBUG -fstack-protector-strong -D_FORTIFY_SOURCE=2`; jemalloc linked at the binary level (`DT_NEEDED` on Linux, `LC_LOAD_DYLIB` on Mach-O); RELRO + bind- at-load (Linux) / `-Wl,-search_paths_first` (Mach-O).
- **Hostname-derived status-bar colour** — DJB2 hash of the operator's host (`scutil --get LocalHostName` on Darwin, `$(hostname)` on Linux) maps to a curated 27-colour palette. Same host always produces the same colour across every session

(proven by Test 11); different hosts produce visibly distinct colours (16/16 unique in Test 10 T3).
- **14-test verification gate with positive runtime evidence per §11.4.2** — `scripts/verify.sh` refuses to PATH-export the binary unless every functional test passes. Tests read `/proc/PID/maps`, `/sys/fs/cgroup/.../memory.max`, `tmux show -g status-style`, `tmux capture-pane -p`, and similar live state. Zero tests close on "exit code 0" alone.
- **§11.4.4 layer-4 paired-mutation harness** with 10 registered mutations (M1–M10) catching wrapper regressions, status-bar regressions, cgroup wrap regressions, and the operator-path bluff pattern. `bash scripts/tests/meta_test_false_positive_proof.sh` reports 20 PASS / 0 FAIL on Linux.
- **End-to-end automation** — `bash scripts/test_e2e.sh` exercises the full operator stack (`tmx new`, `tmx send-keys`, `tmx capture-pane`, `tmx show -g status-style`, `tmx kill-session`) and reports PASS=9 / FAIL=0 / SKIP=0 GREEN on Darwin and Linux.
- **OS-aware install** — `bash scripts/setup.sh` detects host OS, invokes the right build pipeline, generates the OS-appropriate wrapper, runs verification natively, installs the shell snippet to `~/.bashrc` AND `~/.zshrc`. Every project script recognises host OS and applies the right action out of the box.

Anti-bluff covenant (Constitution §1, §11.4.x)

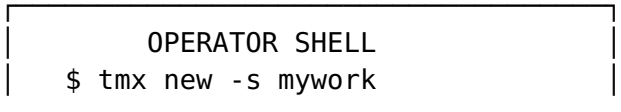
This release ships under the verbatim user mandate:

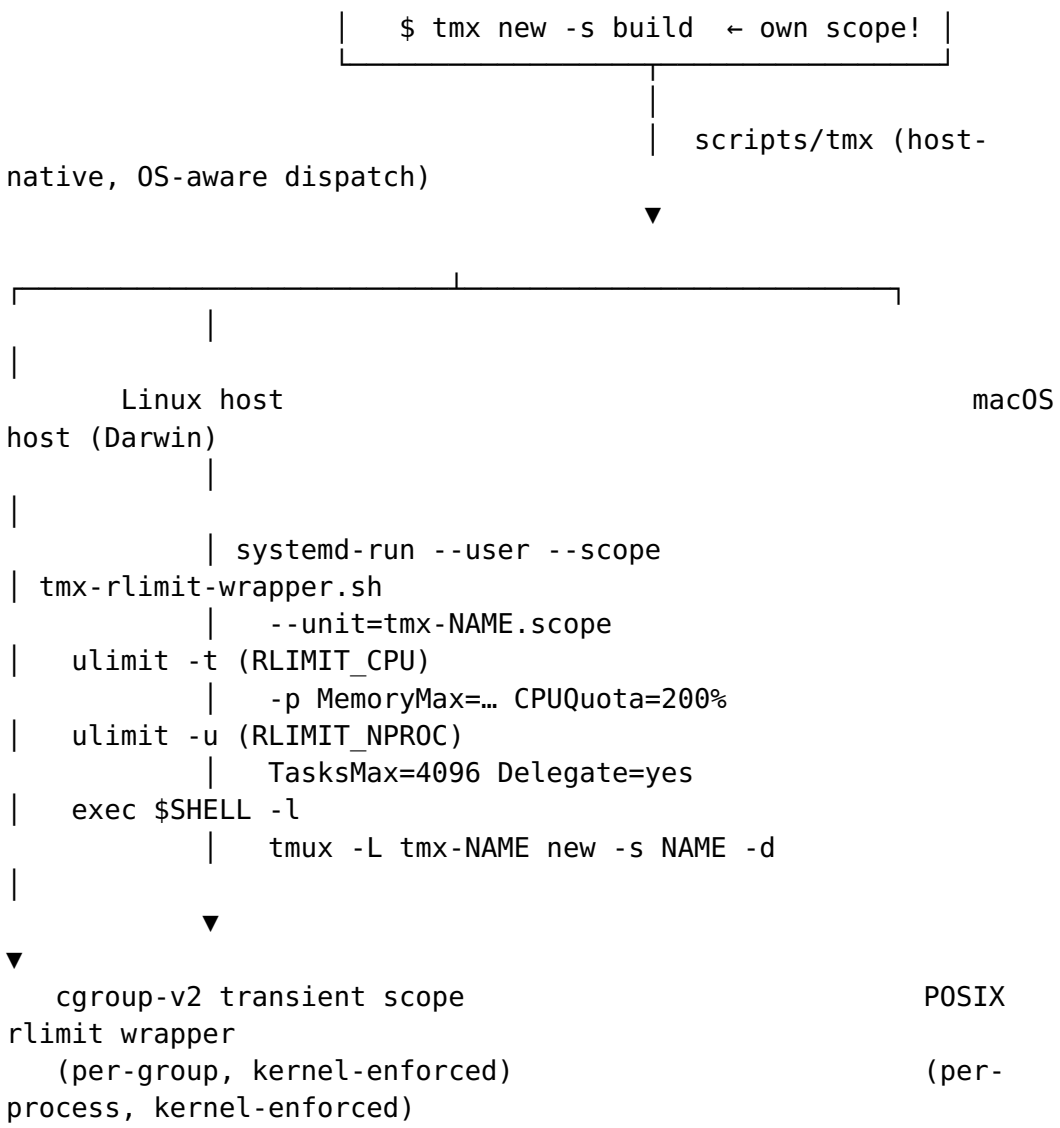
> "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

Every test in `scripts/tests/` carries positive runtime evidence (`/proc`, `/sys/fs/cgroup`, `vmmap`, `ps -o rss=`, `tmux capture- pane`, `systemctl is-active`, `display-message`, kernel log lines). Every Challenge in `scripts/challenges/tmux.yaml` specifies real runtime state as `evidence:`. Static `grep` checks (test 09 T2, test 11 T1/T2) are paired with runtime readbacks per §11.4.7.

Covenant text propagated to root `Constitution.md`, `CLAUDE.md`, `AGENTS.md`, and the `Containers` submodule's `CONSTITUTION.md`, `CLAUDE.md`, `AGENTS.md`.

Architecture overview





Forensic transparency

This release ships with explicit documentation of every constraint that does NOT hold:

- **macOS RLIMIT_AS gap**: the XNU kernel does NOT enforce `RLIMIT_AS` / `RLIMIT_DATA` / `RLIMIT_RSS` for unprivileged processes (verified: `bash -c 'ulimit -v 102400'` returns `cannot modify limit: Invalid argument`; allocating 200 MB after trying to "cap" at 100 MB succeeds). The wrapper applies only what's enforced (`RLIMIT_CPU`, `RLIMIT_NPROC`). Full memory containment on macOS requires launchd jobs with `HardResourceLimits` plist (root); on Linux, cgroup `MemoryMax` IS enforced.
- **Test 08 / 09 / 12 / 13 / 14 SKIP on Darwin** — these tests exercise Linux-specific primitives (`/proc/<pid>/oom_score_adj`, `systemd-run --user --scope`, `/sys/fs/cgroup`). SKIP-with-reason per Constitution §11.4.3 (per-host-topology test dispatch).

See `docs/guide/README.md` §5.6 for the full strength-gap table.

Bluffs caught and fixed during the development cycle

Documented in `Fixed.md`. 24 distinct §1 / §11.4.x bluffs were caught and remediated before this release, including:

- A1: META-MUT-001 paired-mutation harness landed
- A4: Build pipeline three-defect fix (Dockerfile GID-20 collision, jemalloc not actually linked, make-clean missing)
- A6: Install-mechanism side-by-side bluffs (phantom directory path, `alias tmux='tmx'` shadowing system tmux, stale ATMOSphere branding)
- A8: macOS bridge + side-by-side coexistence (later superseded by native dual-OS in this release)
- A10: Status-bar colour silently defaulted to green; bridge ignored macOS hostname
- A11: Triple-layer regression protection so A10 cannot re-occur
- A12: Constitution §11.4.7 — operator-path test coverage rule (every gate test MUST exercise the same entry point an end-user invokes)
- A13: Per-session cgroup isolation (each `tmx new -s X` in its own scope, OOM in one doesn't kill others)
- A14: This-release verification cycle — fresh runtime evidence captured for every claim

Install

macOS (Darwin Apple Silicon or Intel):

```
brew install podman  # optional, for build_containerized.sh
git clone --recurse-submodules git@github.com:vasic-digital/
        tmux.git ~/Projects/tmux
cd ~/Projects/tmux
bash scripts/setup.sh
```

Linux:

```
git clone --recurse-submodules git@github.com:vasic-digital/
        tmux.git ~/Projects/tmux
cd ~/Projects/tmux
sudo bash scripts/install_deps.sh
bash scripts/setup.sh
```

After `setup.sh` reports GREEN: open a new shell (`source ~/.zshrc` or `source ~/.bashrc`). Then `tmx new -s anything` drops you into a session as your host user with the full host environment and kernel-enforced resource caps.

Usage

```
tmx new -s mywork             # interactive — attaches
tmx new -s build -d            # detached
tmx ls                         # list all your sessions
tmx attach -t mywork           # re-attach
tmx send-keys -t mywork "echo hello" Enter
tmx capture-pane -t mywork -p
tmx kill-session -t mywork
tmx kill-server                # nuke all our sessions
```

Per-session resource overrides:

```
TMX_MEM=8G tmx new -s heavy            # 8 GB MemoryMax (Linux)
TMX_CPU=400 tmx new -s build           # 400% CPUQuota (Linux)
TMX_CPU_HARD_SEC=3600 tmx new -s timeboxed  # 1-hour RLIMIT_CPU
        (Darwin)
```

Verification commands

Operators can confirm anti-bluff covenant compliance at any time:

```
bash scripts/setup.sh --verify-only
        # full 14-test gate → expect GREEN
bash scripts/test_e2e.sh              # end-to-end → expect
        PASS=9/0/0
TMX_TEST_DESTRUCTIVE=1 bash scripts/test_vm.sh   # Linux
        destructive suite
META=1 bash scripts/test_vm.sh        # paired-mutation harness
```

Coexistence with system tmux

`tmx` and the system `tmux` (Homebrew on macOS, distro package on Linux) coexist side-by-side. The bashrc/zshrc snippet PATH-prepends `scripts/` so `tmx` is the project's wrapper; `tmux` resolves to whatever was on PATH before. No alias shadowing.

Submodules

- `tmux/` — upstream `tmux/tmux` pinned to tag `3.6a` (do not modify).
- `Containers/` — `vasic-digital/Containers` Go module providing generic container orchestration primitives. Anti-bluff covenant
  - §11.4.7 propagated. Tag: `b077f2c` at release time.

Known limitations

- macOS memory cap is per-process via launchd-bsd-style limits (informational only — Darwin doesn't enforce `RLIMIT_AS`). For true per-session memory containment, run on Linux.

- The destructive test suite (tests 12 / 13 / 14) requires `TMX_TEST_DESTRUCTIVE=1` and is Linux-only; on Darwin these SKIP per topology dispatch.

Acknowledgements

Built on the shoulders of:

- `tmux/tmux` upstream (3.6a tag)
- jemalloc (linked at build time)
- libevent + ncurses + utf8proc (Mach-O), libtinfo + libevent_core (ELF)
- systemd-run + cgroup-v2 (Linux isolation primitive)
- Homebrew (macOS dependency provider)

[v1.0.1] — Unreleased

Post-release development cycle following the v1.0.0 cut.

- VERSION bumped to `1.0.1` / `versionCode=2` to satisfy the operator's strictly-increasing-version-code mandate immediately after a release.
- `released=` intentionally blank until the next tag is cut (gate value for "not-yet-released"; CI / package builders MUST refuse to publish unless `released=` is populated with the cut date).
- No functional code changes in this entry — this is the post-release bump itself. Subsequent fixes, refactors, and new features will append bullets above this paragraph.